lsqvce 6.1.2 — User Manual

least-squares adjustment with variance component estimation

built from the sources; the single documentation authority

Contents

1 Overview **2**
- 1.1 What it does . . . 2
- 1.2 The three sentences you need . . . 2
- 1.3 Reading order . . . 3

2 Installation **3**
- 2.1 Requirements . . . 3
- 2.2 Install . . . 3
- 2.3 Check . . . 3
- 2.4 Uninstall . . . 3
- 2.5 Layout of the toolbox folder . . . 3

3 Declaring the data **3**

4 Options — the complete reference **17**
- 4.1 What to estimate . . . 17
- 4.2 The loop . . . 17
- 4.3 Run control . . . 18
- 4.4 The groups . . . 18
- 4.5 Retired knobs . . . 20

5 The user tier: fit, Options, Results, report, layout **20**
- 5.1 lsqvce.fit . . . 20
- 5.2 lsqvce.Options . . . 23
- 5.3 lsqvce.Results . . . 26
- 5.4 lsqvce.report . . . 28
- 5.5 lsqvce.layout . . . 30

6 Regularisation: priors and groups **31**
- 6.1 The rule . . . 31
- 6.2 What a prior is . . . 31
- 6.3 The declaration — one mechanism for common and local parameters . . . 31
- 6.4 How they are treated . . . 32
- 6.5 The datum of a free network . . . 32
- 6.6 The layout . . . 32
- 6.7 Your table’s column 11 . . . 32

7 Stochastic models and their estimation **32**
- 7.1 7.1 The division of labour — three facts, three places . . . 33
- 7.2 7.2 `Cep` — the three shapes, and the asymmetry that bites . . . 33
- 7.3 7.3 The temporal laws . . . 33

7.4 7.4 The two policy fields on a spec . . . 34
7.5 7.5 What `Estimate="noise"` does . . . 35
7.6 7.6 Worked example — kinematic orbits with a 3×3 per epoch, KBR with one variance 35
7.7 7.7 Reading the result back . . . 36
7.8 7.8 Robustness: what Ellmer does, and what `Robust` adds inside the loop . . . 36

8 Diagnostics and the verdict **37**

9 Watching a run **38**

10 The engine, kernel and util tiers **38**
10.1 lsqvce.engine — drivers, accumulator, loaders, sweeps, diagnostics . . . 38
10.2 lsqvce.kernel — the numerical kernels . . . 39
10.3 lsqvce.util — plumbing . . . 40

11 Theory **40**

12 Migration from 4.x to 5 **40**
12.1 The migration table (from RELEASE_NOTES.md) . . . 40

13 Appendix A — error and warning identifiers **41**

1 Overview

lsqvce is a MATLAB toolbox for least-squares adjustment with variance component estimation, built for satellite gravity recovery (GRACE / GRACE-FO) and equally at home on a levelling network. Version 5 (September 2026) is the result of the R5 review track: one accumulator, one stochastic-model estimator, one loop with three policies, one user surface.

1.1 What it does

- **Adjusts** a linear or linearised Gauss–Markov model with a common parameter block and per-system, per-day local blocks eliminated on the fly (the block-arrow structure of satellite gravimetry), on one accumulator with a QR or normal-equation backend.
- **Estimates the stochastic model** of every observation stream jointly (Ellmer 2018): a spectral shape per pool group, a level per stream, optional per-arc variance factors, by Hutchinson-probed redundancies – `Estimate="noise"`.
- **Regularises by declaration**: priors on common and on local parameters are pseudo-observation groups in your columns, counted in the redundancy, with their sigma fixed or estimated by Helmert / Koch–Kusche.
- **Diagnoses**: Baarda / Pope data snooping, redundancy numbers, reliability, the whiteness of every stream, a five-level verdict (`R.report`).
- **Robust** M-estimation (IRLS) and damped Gauss–Newton on the same loop.
- **Watches**: `Options.OnPass` fires after every pass with the state and the live accumulator; `R.plot` afterwards.

1.2 The three sentences you need

```
lsqvce_setup                                        % once
R = lsqvce.fit(data, Estimate="noise", Pool="obsType");   % the one verb
R.report                                            % the verdict
```

`data` is a plain struct (chapter 3); everything about *how* to run is a `lsqvce.Options` (chapter 4); everything that comes back is a `lsqvce.Results` (chapter 5). The theory behind every number is `lsqvce_theory.pdf` (chapter 11).

1.3 Reading order

New here: chapter 2 (install), then `demo_workflow` next to chapter 3, then chapter 4 when you want to change a knob. Migrating from 4.x: chapter 12 first.

2 Installation

2.1 Requirements

MATLAB R2024a or later (developed and gated on R2026a). The Parallel Computing Toolbox is optional (`Parallel.Mode="parfor"|"spmd"`, `Parallel.DistributeDays`); without it the toolbox runs serially and says so once per run.

2.2 Install

```
cd <toolbox root>          % the folder holding +lsqvce
lsqvce_setup               % or lsqvce_setup('install')
```

This adds exactly two path entries (the root with the `lsqvce.*` packages, and `examples/`), builds the Help browser index so that `doc lsqvce` opens this manual, saves the path, and prints a check report. Not added by design: `dev/tests` (the suites manage their own path), `template/` (you copy it into your project), `dev/`.

2.3 Check

```
lsqvce_setup('check')      % version, path entries, doc index, whether lsqvce.fit resolves
```

2.4 Uninstall

```
lsqvce_setup('uninstall')  % removes the two path entries, saves the path, deletes the
                           % Help browser database it built; the folder itself is untouched
```

2.5 Layout of the toolbox folder

folder / file	what
`+lsqvce/`	the package: user tier (`fit`, `Options`, `Results`, `report`, `layout`), `+engine`, `+kernel`, `+util`
`examples/`	the demos (`demo_workflow`, `demo_fit`, `demo_grace`, `demo_gravity`, `demo_gravfire`, ...)
`template/`	`gravfire_data.m` (a GRACE bundle to the data struct), `lsqvce_template.m`
`html/`	this manual (`lsqvce_manual.html`) and one page per public unit (`doc lsqvce.<name>`)
`lsqvce_theory.pdf`, `lsqvce_manual.pdf`	the theory and this manual as PDF
`RELEASE_NOTES.md`	what changed, with the migration table
`dev/`	the development record (`HANDOVER.md`), the test suites, the documentation sources

3 Declaring the data

This chapter is the binding data contract of the toolbox (formerly the file `DATA_CONTRACT.md`, retired 2026-09-04: the Manual is the single authority). Everything the toolbox knows about your problem enters through the `data` struct described here.

Status. Binding specification, **version 6** (adopted 2026-09-09, Review R6 WP-102: ONE shape changed — `buildDay` returns one CELL per arc, §0c — and the arc schedule became optional; everything the v5 declaration says about tables, priors and `Cep` is unchanged. Option names are those of `lsqvce.Options` v5, the v5 → v6 map is in `RELEASE_NOTES.md`). `lsqvce.fit` is live for all three data kinds, `Estimate = "none" | "noise"`, `Robust`, the nonlinear form and `OnPass`. §9 shows the equivalent direct engine calls. This file is the source of the `fit` help text and of the HTML guide: one authority, no second copy.

Created 2026-08-31 (CEST, UTC+02:00) with assistance from Claude (Anthropic), model Claude Fable 5. Rewritten for v3 and again for v4 (scoped tables) the same day (Claude Fable 5); v6 (the cells are the arcs) 2026-09-09, Claude Fable 5.1.

3.0.1 0a. The v5 delta (R5 M5): three additions, no shape changes

1. **`data.priors(k) = struct('group' | 'L', 'x0', 'sigma', 'from', 'name', 'estimate')`** — declared pseudo-observation groups `L·x ~ x0` with weight `1/sigma²` on the COMMON parameters, folded into the solve AND into the diagnostic sweep (they carry a redundancy). A free network's datum is the prior `L = B'` on the null space: the exact minimum-constraint solution, for any weight, because the data's range and the null space are orthogonal (`lsqvce:fit:badPrior` on malformed fields). A Kaula rule or a background model is the same declaration with a scientific `L`. With `estimate = "scale"` (R5 M7 7b) the group has its OWN variance component: under `Estimate="noise"` its sigma is updated every pass by the Helmert / Koch–Kusche rule ($\Omega_k = \|(L\,x - x0)/\sigma\|^2$, $r_k = p - \mathrm{tr}(L\,N^{-1}\,L')/\sigma^2$, $\sigma^2 \leftarrow \sigma^2 \cdot \Omega_k / r_k$; the fixed point $\Omega_k = r_k$ is "the prior's weighted residual sum equals its redundancy"), so $\lambda = s0^2/\sigma_k^2$ is the regularisation the data support; `R.noise.priors` and `R.noise.priorHistory` record it. `from` is the pass at which the group becomes active (default 1; a single-pass run reports `lsqvce:fit:priorNotActive` and skips it). A group is declared EITHER by `group` — a regular expression on the NAME column of `commonParams` (`data.paramColumns.name`, default 2); fit builds L, one row per matched row per instance — OR by `L` (p × nc in the columns of A); never both (R5-D8 round 6). One `sigma` per group: the rows of L are the members, and several rows are what make `estimate="scale"` estimable (a single row has redundancy ≤ 1). 1b. **`data.systems(k).priors(l) = struct('group' | 'L', 'x0', 'sigma', 'from', 'name', 'estimate')`** (R5-D8 round 5/6) — the priors on system k's LOCAL parameters, the literal counterpart of `data.priors(j)`: one struct per GROUP, declared EITHER by **`group`** — a regular expression matched against the NAME column of the table (column 2; `data.paramColumns.name` to say otherwise); the toolbox builds L with one row per matched template row per instance — OR by **`L`** (q × nl in the columns of that system's B as **`lsqvce.layout(data)`** announces them, used as is); never both. Then `x0`, ONE `sigma` per group (relative weights inside a group go into L), `from`, `name` (default the pattern), `estimate = "none" | "scale"`. Every row must lie inside one day's local block (couple days through common parameters); the width of L is checked by name (`lsqvce:fit:badPrior`). Each group rides as its own observation stream `prior_<name>` of the system (pseudo-observations A = 0, B = L/σ, l = x0/σ, unit law), so accumulation, elimination, the sweeps, `residuals()` and the redundancy count treat it as data — and with `estimate = "scale"` its σ is estimated every pass of `Estimate="noise"` as that stream's level, the Helmert / Koch–Kusche factor the joint estimator computes for every stream: IDENTICAL behaviour to the common groups (`R.noise.model{k}.sigma` for stream `satKey|prior_<name>`). Common priors and every system's priors carry their own settings. THE RULE (R5-D8): the parameters and their columns are yours (tables, `buildDay`, plots, `toTable`); regularisation is the toolbox's (mechanism, redundancy, activation, VCE per group); the interface between them is an operator in your columns, announced by `lsqvce.layout`, checked by width. `Options.Reg / RegLambda / RegMatrix` are gone. 1c. **`data.paramColumns = struct('start', c1, 'end', c2)`** (R5-D8 round 3) — names the columns of BOTH tables that hold a row's validity (`[MJD seconds]` pairs or fractional MJD; `[]` = always). A `'day'` / `'arc'` row is then instantiated only on the calendar days its validity overlaps (half-open day: start < d+1 and end > d); `'common'` / `'system'`

rows are always present. nc and nl are derived accordingly, and `buildDay`'s 6th argument tells you which template rows exist on the day: `info.arcRows`, `info.dayRows`, `info.sysRows`, `info.commonDayRows` (indices into the tables, in block order). Without `paramColumns` every row is valid always — the v4 behaviour. This is how a one-day manoeuvre parameter and a "literal" row become the same mechanism as everything else.

2. **Per-stream policy on the covariance spec** — `covariance.<type>.estimate = "none" | "scale" | "law"` (default `"law"` under `Estimate="noise"`) and `.arcFactors` (logical, default the Options policy). A law whose `sigma` is a VECTOR (per-row weights) is a fixed structure: it can carry a level (`"scale"`) but not a spectral shape (`"law"` is refused, `lsqvce:fit:lawOnFixedStructure`). `Options.Pool` decides which streams SHARE one estimated spectral shape; every stream keeps its own level. To read an arc's standard deviation back, use all three terms: **declared sigma × sqrt(arc factor) × `R.sigma0`** — the level and the factor split one scale by a convention (eq. 6.5.30's `gammaBar`) and `sigma0` carries the rest, as in any GLS adjustment; measured on a fixture planted at 0.5 / 1.5 / 0.5, that reconstruction gives 0.517 / 1.499 / 0.507 (R6 WP-100). (R6 finding F-01, fixed in WP-98: eq. 6.5.28 multiplies by the factor the weighting used, so the ratio now settles across passes — 9.4 / 8.6 / 8.6 / 8.6 against a planted 9, where before it grew by 8.4x every pass. Runs with `arcFactors = false` are unchanged.)
3. **The per-pass inspection hook** — `Options.OnPass = @(ctx)` receives, after every pass of every iterative run, the pass state, the live accumulator, `ctx.data` (this declaration as given) and `ctx.processingOptions{k}` (system k's `processingOptions` as declared); return `false` to stop. Nothing else about `buildDay` or the tables changed.

Semantics made explicit in v5: a full SPD weight matrix returned by a batch source is whitened exactly (`chol(P)`), and per-observation diagnostics of a batch problem are reported in the WHITENED frame (`R.diag.v = sqrt(p).*v`; only their sums are frame-invariant).

3.0.2 0b. The v5.3 delta (R6 WP-90): the error information travels with the day

What the preprocessing knows about the accuracy of every observation — a per-epoch standard deviation, per-axis standard deviations, or a full 3 × 3 covariance per epoch — is DATA, and it comes back from `buildDay` WITH the day, as **`D.Cep`** (§3.4, §7), for ANY stream: `nEpoch × 1` for a scalar stream, `nEpoch × d` or `nEpoch × d × d` for a multi-axis one. The axis count is derived from it. The covariance spec (§5) declares the TEMPORAL law only — `diag` (white), `arma`, `psd`, `acf` — and the toolbox composes C = D (R_t ⊗ I_d) D per arc. Retired with this: the `separable` type (it was `arma` + Cep), the `epochCovariance` hook (an unknown field now), and a static `Cep` on the spec (refused when the day carries one: one fact, one authority). Nothing else about the declaration changed.

3.0.3 0c. The v6 delta (R6 WP-102): the cells are the arcs, and the schedule is optional

`buildDay` returns every field as a **1 × nArc cell, one cell per arc of the day**: `A`, `B`, `l`, `t` and, when given, `Cep` (§3.3, §3.4). A day with one arc returns `{A}`; a day without data returns empty cells. The v5 row-index fields `arcIdx` / `arcId` are gone — the cells say what they said — and an **empty cell is a data gap** (§4.3). The driver was the GRAVFIRE preprocessing, which delivers every day as a cell array, one cell per noise-independence piece: the day's cells now ARE the arcs, with no bookkeeping in between.

`arcs` is **optional**. Without it, the arcs of a day are its cells (ids 1 … nArc) and the toolbox learns their number when the engine's per-day `meta` probe reads the day — which the engine reads right after, so the day cache makes the discovery free (`CacheDays` counts DAYS: one resident day is one slot per stream). `'day'` template rows then live on **every** calendar day, and a calendar day that returns no arc in any stream is refused at its probe (`lsqvce:makeLoader:emptyDay`) — take it off the calendar, or declare a schedule. A schedule is **required** only by `'arc'` template rows (nl counts one block per (day, arc) and must be known before any day is read); declared, cell a is the a-th scheduled id and the count must match (`lsqvce:makeLoader:arcCount`). The v5 string form `'fromDay'` is retired: omitting `arcs` is what it meant.

3.0.4 0. The v4 principle: state nothing the data already states — and scope, don't number

The contract declares **structure**, never **counts**, and since v4 it declares **scope, never numbering**: column 1 of both parameter tables is a **scope token**, the rows are **templates**, and the toolbox expands them over the calendar. Every count is derived:

quantity	derived from
nc (common unknowns)	the expansion of `data.commonParams` (`'common'` rows once + `'day'` rows per calendar day with arcs)
nl (local unknowns)	the expansion of `localParams` (`'arc'` rows per (day, arc) + `'day'` rows per day with arcs + `'system'` rows once); **not declared → nl = 0**
observation types	`fieldnames(covariance)`
u (batched nonlinear)	`numel(x0)`

The v2 count fields are gone (v3), and so is v3's hand-numbering split (`localsPerArc` XOR free `localParams`): **one mechanism**, the scoped table. Every column offset reaches `buildDay` through its 6th argument (`info`, §6.3) — no offset is ever computed in user code. The driver was the GRACE parameterization: arc-wise ICs, day-wise ACC scale/bias/drift, per-arc KBR empiricals — three scopes, one table.

3.0.5 1. The mental model in one paragraph

You describe **one adjustment problem** with a plain MATLAB struct called `data`. It answers three questions: *what is estimated for everybody* (`data.commonParams`, one row per common unknown), *which days are processed* (`data.days`), and *which observing systems deliver data* (`data.systems`, one entry per satellite or instrument). Each system points to **one function you write**, `buildDay`, that returns the design matrices and observations of a single day. Everything else — reading days in the right order, cutting them into arcs, whitening with your covariance model, iterating the variance components — is the toolbox's job:

```
opts = lsqvce.Options(Estimate="noise");  % what to do
R    = lsqvce.fit(data, opts);           % the one verb
R.report                                 % the verdict
```

You never call `makeLoader` yourself; `fit` builds the loaders from `data`. `R.systems` (the existing Results field) records what ran: per system its id, nl, the loader closure, the day list and the name.

3.0.6 2. The `data` struct — field by field

`data` is a **scalar struct**. Field names are **case-sensitive** and use lowerCamelCase. Unknown field names are an **error** (`lsqvce:fit:unknownField`), which is deliberate: a typo in a struct field would otherwise be silently ignored. The retired v2 names (`common`, `numCommon`, `numLocal`, `observationTypes`, `systemId`, `cache`, `loader`, the plot hooks) are unknown fields now — the error names the field, so a v2 declaration fails loudly, not subtly.

3.0.6.1 2.1 `data.commonParams` — the shared unknowns (required)

field	type / size	meaning
`commonParams`	cell, R×C (C ≥ 1)	one row per common **template**. **Column 1 is the scope token**: `'common'` (one column of A, always) or `'day'` (one column per calendar day that carries at least one scheduled arc in at least one system). Columns 2…C are *your* convention (degree, order, epoch, …). nc is **derived** from the expansion; the table is piped to `buildDay` as its 3rd argument. `'arc'` is forbidden here — arc sets are system-specific (`lsqvce:fit:badScope`).

Binding column layout of A: the `'common'` rows first (table order), then one day block per calendar day (calendar order — the sorted union of every declared day list). `buildDay` receives this day's block position as `info.commonDayOffset` / `info.commonDayCount` (§6.3). Minimal table when you have nothing to say: `[repmat({'common'}, nc, 1), num2cell((1:nc).')]`.

3.0.6.2 2.2 `data.days` — the default schedule

field	type / size	meaning
`days`	double, 1×D	day identifiers, e.g. `1:30` or `datenum` values cast to integers. Every system uses this list unless it carries its own `days`. Required unless every system has one.

Days must be **numeric integers**: the loader cache is keyed on the number.

3.0.6.3 2.3 `data.systems(k)` — one entry per observing system

`data.systems` is a **1×M struct array** (M ≥ 1). All entries have the same fields; fill an unused optional field with `[]`.

Required — two fields

field	type / size	meaning
`buildDay`	function handle	your day builder, §3.
`covariance`	struct	one field per observation type holding a covariance spec (§5), e.g. `cov.kbr = struct('type','diag','sigma',0.2)`. **The fieldnames ARE the observation types** (their order is the processing order); `buildDay` is called once per (day, type).

Optional

field	type / size	default	meaning
`arcs`	double or fh	*(none)*	the arc schedule, §4: omitted, the cells `buildDay` returns are the arcs of the day; required only by `'arc'` template rows.
`localParams`	cell, R×C (C ≥ 1)	`{}`	the local **templates**, §6: column 1 is the scope token `'arc'` | `'day'` | `'system'`, columns 2…C your convention; nl is **derived** from the expansion. Passed to `buildDay` as its 4th argument. Not declared → **nl = 0** (§6.4).
`days`	double, 1×D	`data.days`	per-system schedule override.
`id`	double 1×1 **or** single char	the index k	system identifier, unique across systems (explicit ids and index defaults share one namespace). Longer labels go in `name`.
`name`	char / string	`''`	human-readable label used in reports.
`processingOptions`	struct	`struct()`	anything your `buildDay` needs (paths, force models); passed as its 5th argument.

Rule of thumb for beginners: `buildDay` + `covariance` is a complete system — return one cell per arc and the arcs are declared. Add `'arc'` template rows in `localParams` when arcs carry their own unknowns (and then `arcs`, the schedule those rows need), the rest when you need it. (The v5.2 `epochCovariance` hook is gone — the per-epoch error information comes back from `buildDay` as `D.Cep`, §7.)

3.0.6.4 2.4 A pitfall you will hit once: `struct()` with cell values

```
% WRONG: struct() expands cell values into a struct ARRAY
s = struct('localParams', {'bias','drift'});          % s is 1x2
% RIGHT: double braces, or assign field by field
s = struct('localParams', {{'bias';'drift'}});        % s is 1x1
s.localParams = {'bias'; 'drift'};                    % clearest
```

3.0.7 3. The `buildDay` contract

3.0.7.1 3.1 Who calls it, and how often `fit` calls `buildDay` **once per (day, observationType)**; the cells it returns are the day's arcs. Every pass over the data reads each day **twice** (once to accumulate the normal equations, once for the residuals), and the variance-component loop multiplies that by the number of passes. Two consequences:

1. **Determinism is mandatory.** Same arguments → byte-identical output. Random numbers, clocks, mutable globals are forbidden inside `buildDay`. (The residuals are formed as `v = l − A·x − B·y` from the second read; any difference lands in σ_0.)
2. **Cache expensive reads yourself.** The toolbox caches *one built day* (every stream of it; `Options.CacheDays` for more); if the raw file read is the expensive part and serves several observation types, keep your own cache in the closure that binds `buildDay`.

3.0.7.2 3.2 Signature — declare only what you need `fit` passes as many of these six arguments as your handle **declares** (`nargin`):

```
D = buildDay(day, obsType)                                  % classic
D = buildDay(day, obsType, common, local, procOpt)          % parameter-driven
D = buildDay(day, obsType, common, local, procOpt, info)    % with the expansion info
```

argument	what it is
`day`	one entry of the day list (numeric)
`obsType`	one entry of `fieldnames(covariance)` (char)
`common`	`data.commonParams`
`local`	`localParams` of this system (or `{}`)
`procOpt`	`processingOptions` of this system
`info`	this day's expansion slice — every offset of A and B (§6.3); `[]` only when nothing is scoped

A `varargin` builder receives all six.

3.0.7.3 3.3 Return value — required fields: THE CELLS ARE THE ARCS Every field is a **1 × nArc cell**, one cell per arc of the day; cell a is arc a. A day with one arc returns `{A}`, `{B}`, `{l}`, `{t}`; a day without data returns empty cells (nArc = 0).

field	size	meaning
`A`	cell, 1×nArc; each m_a×nc	design matrix of the **common** unknowns of arc a
`B`	cell, 1×nArc; each m_a×nl	design matrix of the **local** unknowns; may be `sparse`
`l`	cell, 1×nArc; each m_a×1	**observed minus computed** (the reduced observation). Not the raw observation.
`t`	cell, 1×nArc; each m_a×1	epoch of each row in **seconds** (the whitener needs the lags)

Arc lengths may differ freely; the whitener is built per arc. An arc whose four entries have **0 rows is a data gap** (§4.3) and is skipped. Return **raw, unwhitened** matrices. Whitening happens per arc inside the toolbox from your covariance spec, and the VCE loop reweights there too — a builder that scales its own rows would be reweighted twice.

3.0.7.4 3.4 Return value — optional fields

field	size	meaning
`Cep`	cell, 1×nArc; each nEpoch_a×1, nEpoch_a×d or nEpoch_a×d×d	**the error information of each arc** (§7): the per-epoch standard deviation of a scalar stream, the per-axis standard deviations, or the full per-epoch covariance of a multi-axis stream. nEpoch_a·d must equal the arc's row count; the axis count d is derived from it and is one for the day.
`nAxis`	1×1	the axis count of a multi-axis stream WITHOUT `Cep` (rows epoch-major, m_a = nEpoch_a·nAxis, §7). Derived from `Cep` when that is present; given as well, it must agree.

No other field is allowed — `arcIdx` / `arcId` (the v5 way of saying what the cells say) or a misspelt `cep` are refused by name (`lsqvce:makeLoader:badDay`), so a typo cannot be ignored silently.

3.0.7.5 3.5 What is checked, and the error you get

error id	meaning
`lsqvce:makeLoader:buildDay`	your `buildDay` threw; the original error is attached as the cause
`lsqvce:makeLoader:badDay`	a required field (`A`, `B`, `l`, `t`) is missing or is not a 1×nArc cell, or the day carries a field that is not `A B l t Cep nAxis`
`lsqvce:makeLoader:dayShape`	the cells do not agree in number, or an arc's `A`, `B`, `l`, `t` do not share a row count
`lsqvce:accum:arcBadNc` / `arcBadNl`	`A` or `B` has the wrong number of columns — the derived nc/nl are enforced against **every** arc
`lsqvce:makeLoader:arcCount`	a declared schedule names n arcs for the day and `buildDay` returned a different number of cells
`lsqvce:makeLoader:emptyDay`	without a schedule, a calendar day that owns a `'day'` block returned no arc in any stream
`lsqvce:makeLoader:badArc`	an arc id outside the declared schedule was asked for (a static schedule declared wrongly), or an arc of a multi-axis day does not hold whole epochs
`lsqvce:makeLoader:badDayCep`	`Cep` is not a 1×nArc cell, or an arc's entry is not nEpoch×1 / nEpoch×d / nEpoch×d×d, or nEpoch·d is not the arc's row count
`lsqvce:makeLoader:axisConflict`	`nAxis` disagrees with `Cep` (or with a `d` on the spec), or does not divide the row count
`lsqvce:makeLoader:cepTwice`	the covariance spec carries a `Cep` AND the day returns one

error id	meaning
`lsqvce:covModel:cepOnVar`	a `var` law (its own cross-axis structure) with a `Cep`
`lsqvce:makeLoader:noCep`	*(warning, once per run)* a multi-axis day without `Cep`: the axes are weighted by the temporal law alone (D = I)

These are raised **while reading**, by the engine, and keep their engine ids. Errors in the *declaration* itself are raised by `fit` before any data is read and carry `lsqvce:fit:` ids (`unknownField`, `noSystems`, `noCommon`, `missingField`, `localCount`, `noDays`, `badId`, `duplicateSystem`, `builderKind`, `badArcs`, and every `makeLoader` construction id relabelled, e.g. `lsqvce:fit:badScope`).

3.0.8 4. Arc schedules

3.0.8.1 4.0 None — the cells are the schedule (the default since v6) Omit `arcs`. The arcs of a day are the cells `buildDay` returns, ids 1 … nArc, and days may differ freely in their count. The toolbox learns the count when the engine's per-day `meta` probe reads the day (the engine reads it right after; the cache keeps a whole day resident, so nothing is read twice). Consequences:

- `'day'` template rows (§6) and `'day'` rows of `commonParams` are instantiated on **every** calendar day of the system — the calendar is the declaration.
- A calendar day that returns **no** arc in any stream is refused at its probe (`lsqvce:makeLoader:emptyDay`) before any solve could go rank-deficient on an unobserved block. Take the day off the system's calendar (`days`), or declare a schedule that names no arc for it.
- `'arc'` template rows need a schedule (§4.1 / §4.2): nl counts one block per (day, arc) and must be known before any day is read (the `localPlan` construction error otherwise).

3.0.8.2 4.1 Static — every day looks the same

```
s.arcs = 4;            % arcs 1..4 on every day, every observation type
s.arcs = [1 2 4];      % these ids on every day
s.arcs.kbr = 2;  s.arcs.pos = 4;    % per observation type
```

Cell a of the returned day is arc ids(a); the day must return exactly as many cells as the schedule names (`lsqvce:makeLoader:arcCount`), which catches a mis-declared schedule.

3.0.8.3 4.2 Ragged — days differ (gaps, manoeuvres, short months) Declare a **resolver** that returns the arc ids of a day *without reading data*:

```
s.arcs = @(day, obsType) arcTable{day};      % 1xA vector of ids for that day
```

and return one cell per id, in that order. This is the form `'arc'` template rows need: a day whose list is empty gets no local blocks and no common day block (§6.4).

3.0.8.4 4.3 Data gaps — the empty cell Under a declared schedule, an arc the day does not carry is returned as an **empty cell** (0 rows in `A`, `B`, `l`, `t`): every engine skips it, no padding, no error. The same run declared with a resolver that simply omits the id gives bitwise the same result. (An empty cell of an arc that carries `'arc'` template rows leaves those unknowns unobserved — say the gap in the resolver instead.)

The v5 string form `arcs = 'fromDay'` is retired (`lsqvce:fit:badArcs`): omitting `arcs` is what it meant.

3.0.9 5. Covariance specs — the `covariance.(obsType)` structs

One struct per observation type; `type` selects the **temporal law** — how the errors of one stream are correlated down the epochs. The most common:

`type`	fields	use for
`'diag'`	`sigma` (scalar std)	uncorrelated noise, e.g. `struct('type','diag','sigma',0.2)`
`'arma'`	`ar` `[1 a1 … ap]`, `ma` (default 1), `sigma` (innovation std)	coloured noise with a known filter
`'acf'`	`acf` `@(dt,opt)`, `sigma`	correlation given as a function of the lag
`'psd'`	`f` (nf×1, Hz), `S` (nf×1)	noise given as a one-sided PSD
`'var'`	`ar` (d×d×p), `sigma` (d×d)	vector-AR for multi-axis streams with cross-axis temporal correlation (no `Cep`)

`sigma` is always a **standard deviation**, never a variance, and it is the LEVEL of the law. The accuracy of the individual observations — what varies from epoch to epoch — is not declared here: it comes back from `buildDay` with the day as `D.Cep` (§7), and the toolbox composes the two. The VCE loop scales and reshapes these specs at run time; you only give the starting point. Full reference: `doc lsqvce.engine.covModel`.

3.0.10 6. Local unknowns — the scoped `localParams` table

One mechanism: the table's rows are templates, column 1 says **how often** each row instantiates.

3.0.10.1 6.1 The three scopes

```
s.localParams = [ ...
    [repmat({'arc'}, 6, 1),  num2cell((1:6).')];   % 6 ICs   -> once per (day, arc)
    {'arc','kbrBias'; 'arc','kbrCos1cpr'};         % 2 more  -> once per (day, arc)
    {'day', 'accScale'; 'day', 'accBias'};         % 2       -> once per day with arcs
    {'system', 'antennaOffset'}];                  % 1       -> once, full adjustment
```

token	one block per …	typical use
`'arc'`	(day, arc) of this system	initial conditions, per-arc empiricals
`'day'`	day whose schedule carries ≥ 1 arc — every calendar day when no schedule is declared	ACC scale/bias/drift
`'system'`	whole adjustment	instrument constants; also the v3 free table (see migration)

Requires `days` whenever `'arc'`/`'day'` rows are present, and a declared schedule (static or resolver, §4) for `'arc'` rows; `'day'` rows without a schedule live on every calendar day (§4.0); `'system'`-only tables need neither. nl is derived; declaring `numLocal` too is the conflict error.

Binding column layout of B (per system): per day (declared order) — the arc blocks in **sorted arc-id order** (each block = the `'arc'` rows, table order), then the `'day'` block — and after all days one trailing `'system'` block. A day whose declared schedule has no arcs gets **no blocks at all** (§6.4).

3.0.10.2 6.2 Heterogeneous per-arc counts stay ENGINE-tier

A template has the same rows for every arc. If arcs genuinely carry *different* counts, that is the ENGINE-tier `lsqvce.engine.makeLoader(..., 'LocalsPerArc', @(day, arcId) n)` (the WP-52 plan) — deliberately not on the `fit` surface.

3.0.10.3 6.3 Using `info` (buildDay's 6th argument)

```
info.arcId            % 1xA  ids of this day's arcs
info.offsetOf         % @(arcId) -> column offset of that arc's block in B
info.count            % 1xA  columns of each arc block (= number of 'arc' rows)
info.dayOffset        % 1x1  this day's 'day' block offset ([] if none)
info.dayCount         % 1x1  its width (0 if none)
info.sysOffset        % 1x1  the 'system' block offset
info.sysCount         % 1x1  its width
info.nl               % 1x1  total local count of the adjustment
info.nc               % 1x1  total common count
info.commonCount      % 1x1  width of A's leading 'common' block
info.commonDayOffset  % 1x1  this day's common day-block offset in A ([] if none)
info.commonDayCount   % 1x1  its width (0 if none)
```

Fill `B(rows, info.offsetOf(id) + (1:k))`, `A(:, info.commonDayOffset + (1:w))` — and **never compute an offset yourself**.

3.0.10.4 6.4 Days without arcs, systems without locals

A calendar day on which **no system** schedules an arc gets no common day block (nc shrinks); a system day whose declared schedule has no arcs gets no arc and no `'day'` blocks (nl shrinks) — offsets stay gap-safe on ragged schedules and manoeuvre days. A system **without** a schedule claims every day of its calendar (§4.0). A system without local unknowns is still the **default**: declare no `localParams` and return `B = zeros(m, 0)` (the engine skips the pre-elimination; empty diagnostics per HANDOVER G-07).

3.0.11 7. The error information — `D.Cep` on the returned day

The covariance of a stream has two parts. The **temporal law** R_t (white, AR, a PSD) is declared in §5 and estimated by `Estimate="noise"`. The **per-epoch accuracy** D — what the preprocessing knows about each observation: a PPP covariance, a per-epoch sigma from a fit, a weight from an earlier screening — is DATA, and you return it with the day:

```
D.Cep{a} = sigmaPerEpoch;                  % nEpoch_a x 1      scalar stream
D.Cep{a} = [sx sy sz];                     % nEpoch_a x 3      per-axis sigmas (diagonal D)
D.Cep{a} = permute(reshape(Q.', 3, 3, []), [3 1 2]);   % nEpoch_a x 3 x 3  full per-epoch covaria
```

(one entry per arc, a 1×nArc cell like `A`; v6)

The toolbox composes C = D (R_t ⊗ I_d) D per arc: every epoch is de-scaled by D^-1/2 (for d = 1 that is a division by the sigma), then the temporal whitener runs down the epochs of each axis. The axis count d is `size(Cep, 2)`; `nAxis` is derived from it. Rows of a multi-axis stream are **epoch-major** (x_1 y_1 z_1 x_2 y_2 z_2 …), so nEpoch·d must equal the row count, and an arc must hold whole epochs. Under `Estimate="noise"` the estimator de-scales the residuals of every arc by the same D before it forms the lag statistics, so the estimated law is the law of the *normalised* errors — a scalar stream with a per-epoch sigma is handled exactly like a 3-axis one.

Rules:

- One authority: `Cep` lives on the day. A `Cep` on the covariance spec together with one on the day is refused (`cepTwice`); a `d` on the spec that disagrees with the day is refused (`axisConflict`).

- A multi-axis day **without** `Cep` (set `nAxis`) is composed with D = I and reported once per run (`lsqvce:makeLoader:noCep`) — the omission is distinguishable from a choice.
- `sigma` on the spec stays the LEVEL of the law; `Cep` is relative to it. Declare `sigma` 1 when `Cep` already carries the absolute accuracy.
- A `var` law carries its own cross-axis structure and takes no `Cep`.

3.0.12 8. Worked example — the GRACE parameterization: three scopes, one table

Arc-wise initial conditions, **day-wise** accelerometer scale/bias/drift, per-arc KBR empiricals — the case v3 could only hand-number, and the driver of v4:

```
% ---- the one function you write -------------------------------------
function D = buildDayGrace(day, obsType, common, ~, procOpt, info)
    raw = readGraceDay(procOpt.dataDir, day, obsType);   % your reader, deterministic;
                                                         % one entry per arc of the schedule
    nArc = numel(info.arcId);                            % the resolver's ids, in order
    D = struct('A', {cell(1, nArc)}, 'B', {cell(1, nArc)}, 'l', {cell(1, nArc)}, 't', {cell(1, nA
    for a = 1:nArc                                       % THE CELLS ARE THE ARCS (v6)
        arc = raw.arc(a);                                % empty on a data gap -> empty cell
        D.A{a} = shPartials(arc, common);                % m_a x info.nc; the 'common' block
        D.l{a} = arc.observed - arc.computed;            % O - C
        D.t{a} = arc.tSeconds;
        B = sparse(numel(D.l{a}), info.nl);
        off = info.offsetOf(info.arcId(a));
        B(:, off + (1:info.count(a))) = icAndKbrPartials(arc);
        if info.dayCount > 0                             % this day's ACC block
            B(:, info.dayOffset + (1:info.dayCount)) = accPartials(arc);
        end
        D.B{a} = B;
    end
end

% ---- the declaration ------------------------------------------------
data.commonParams = [repmat({'common'}, 3717, 1), shParameterTable(60)];
data.days = 1:31;
cov.kbr = struct('type','arma','ar',[1 -1.6 0.9 -0.25],'ma',1,'sigma',1e-7);
cov.pos = struct('type','diag','sigma',0.02);             % fieldnames = the two types

s = struct();
s.buildDay = @buildDayGrace;
s.covariance = cov;
s.id = 'C';
s.name = 'GRACE-FO C';
s.arcs = @(day, obsType) arcSchedule('C', day);           % required by the 'arc' rows below
s.localParams = [ ...
    [repmat({'arc'}, 6, 1), {'x0';'y0';'z0';'vx0';'vy0';'vz0'}];  % ICs per arc
    {'arc', 'kbrBias'; 'arc', 'kbrCos1cpr'};                      % KBR per arc
    [repmat({'day'}, 3, 1), {'accScale';'accBias';'accDrift'}]];  % ACC per day
s.processingOptions = struct('dataDir', '/data/gracefo');
data.systems(1) = s;

s.id = 'D';  s.name = 'GRACE-FO D';  s.arcs = @(day, obsType) arcSchedule('D', day);
data.systems(2) = s;

% nl per satellite = SUM over its days-with-arcs of (arcs*8 + 3) -- derived,
```

```
% gap-safe on manoeuvre days, never stated.

% ---- the run -------------------------------------------------------------
opts = lsqvce.Options(Estimate="noise", Pool="stream", ArcFactors=false, Parallel=struct('Mode',
R = lsqvce.fit(data, opts);
R.report
```

3.0.13 9. What `fit` does internally — the equivalent direct engine calls

```
L = cell(1, numel(data.systems));
for k = 1:numel(data.systems)
    s = data.systems(k);
    L{k} = lsqvce.engine.makeLoader(s.buildDay, ...
        ObservationTypes=fieldnames(s.covariance).', Covariance=s.covariance, ...
        SystemId=s.id, Name=s.name, Arcs=s.arcs, ...                % Arcs only when declared
        LocalParams=s.localParams, Scoped=true, CommonPlan=plan, ...  % fit builds `plan`
        Days=data.days, CommonParams=data.commonParams, ...          % over the global calendar
        ProcessingOptions=s.processingOptions);
end
R = lsqvce.engine.adjustBlock(L, data.days);                        % Estimate "none"
R = lsqvce.engine.adjustRobust(L, data.days, 'Influence', "huber");  % Robust "huber"
% Estimate "noise" is lsqvce.engine.adjustVCEjoint over an accumulator factory that
% lsqvce.fit builds (jointModelOpt + the block accumulator); use fit for it.
```

These are ENGINE-tier calls (experts); a user needs none of them. The ENGINE tier keeps the v2/v3 conveniences (`numCommon`/`numLocal` counts, free un-scoped tables with `Scoped=false` — the default — `LocalsPerArc` incl. the heterogeneous `@(day,arcId)` form, `cache`, raw 4-arg loader closures) — they left the USER surface, not the toolbox.

Field names map 1:1 to `makeLoader`'s Name-Value names (lowerCamelCase field ↔ UpperCamelCase name); `fit` performs exactly this mapping internally, plus the three derivations of §0.

3.0.14 10. Checklist before the first run

- ☐ `buildDay` returns `A`, `B`, `l`, `t` as one cell per arc, each arc with one row count; `l` is O − C; nothing is whitened; no field beyond `Cep` / `nAxis`.
- ☐ Calling `buildDay` twice with the same arguments gives identical output.
- ☐ `t` is in seconds; the sampling matches the covariance spec you declared.
- ☐ `data.commonParams` rows carry the scope token in column 1; `size(A, 2)` must equal `info.nc` (use `zeros(m, info.nc)` and place blocks by the `info` offsets).
- ☐ Every observation type is a field of `covariance` — the fieldnames are the type list.
- ☐ Arc schedule: none (the cells are the arcs) unless `'arc'` template rows need one; declared, one cell per scheduled id, an empty cell for a gap.
- ☐ Local unknowns: scoped `localParams` templates, `B` filled from `info` offsets only; not declared → `B = zeros(m, 0)`.
- ☐ The error information: `Cep` on the day, one cell per arc (nEpoch×1 for a scalar stream; nEpoch×d or nEpoch×d×d, rows epoch-major, for a multi-axis one); the spec declares the temporal law and its level only.
- ☐ No unknown field names in `data` (a typo is an error, on purpose — and so are the v2 count fields and, since v4, `localsPerArc`).

3.0.15 11. The two in-memory kinds (no local unknowns, diagonal weights)

Not every problem streams. When A fits in memory and there are no arc-wise unknowns, `data` has no `systems` at all — `fit` tells the kinds apart by the fields present:

Kind B — matrices

field	size	meaning
`A`	m×u	design matrix
`l`	m×1	observed minus computed
`p`	m×1	weights (diagonal of P), optional; absent = unit weights

Kind C — batched callback (A too large for one matrix, or naturally batched)

field	size	meaning
`batchFcn`	fh	`[A, l, p] = batchFcn(k)`, k = 1…nBatch — the `lsqvce.engine.adjust` contract
`numBatches`	1×1	nBatch
`numUnknowns`	1×1	u — **linear form only**
`x0`	u×1	optional. With it the problem is **nonlinear**: `[A, w, p] = batchFcn(k, x)` returns the Jacobian at `x` and `w = l − f(x)`; solved by Gauss-Newton (`lsqvce.engine.solveNonlinear`). **u = numel(x0)** — `numUnknowns` must then be absent (`lsqvce:fit:uCount`)

Rules: `opts.Estimate` must be `"none"` (variance components need the streams of a streaming system — `lsqvce:fit:modelKind`); `opts.Robust = "huber" | "igg3" | "bisquare"` switches to M-estimation (`RobustOpts` in Options); shapes are checked before anything runs (`lsqvce:fit:badMatrix`, `lsqvce:fit:missingField`, `lsqvce:fit:unknownField`). Both kinds run on the ONE accumulator as a local-free streaming system (batch k = day k), so `R.residuals()`, `R.contribution()`, `R.plot(..)` and `OnPass` work on them too; the per-observation diagnostics are in the whitened frame (`R.diag.v = sqrt(p).*v`).

What comes back is the same `Results`: `R.x`, `R.sigma0`, `R.coefficients()`, `R.report()`, `R.diag`, and for IRLS / Gauss–Newton the pass record on `R.history` (the final IRLS factors: `R.history(end).factor`) with `R.stopReason`.

```
data = struct('A', A, 'l', l, 'p', p);
R = lsqvce.fit(data);                                   % adjustBlock on the batch adapter
R = lsqvce.fit(data, Robust="huber");                   % adjustRobust (IRLS)
data = struct('batchFcn', @(k, x) myJacobian(k, x), 'numBatches', 20, 'x0', x0);
R = lsqvce.fit(data, MaxPasses=15);                     % adjustNonlinear (Gauss-Newton)
```

Stream names carry no meaning: the axis count of a stream comes from the day it returns (`Cep` or `nAxis`, §7), never from its name.

4 Options — the complete reference

`lsqvce.Options` is a typed value class: every knob has a type, a range and a default, checked at assignment (`MATLAB:validators:*` identifiers) or at construction (`lsqvce:Options:*`). It says *how to run*; everything about the data, including regularisation, is declared on `data` (chapter 3). Three equivalent spellings:

```
opts = lsqvce.Options(Estimate="noise", MaxPasses=6);   R = lsqvce.fit(data, opts);
opts.Noise.Grid = 2160;                                  % change one later (validated)
R = lsqvce.fit(data, Estimate="noise", MaxPasses=6);     % straight on fit
```

`disp(opts)` lists every knob, a * marking the defaults. The option space is **14 top-level knobs and 5 groups**; the count is pinned by a test so it cannot grow unnoticed.

4.1 What to estimate

`Estimate` — `"none"` (default) | `"noise"`. `"none"` is one generalised least-squares solve under the declared covariance laws. `"noise"` runs the joint variance-component estimator (Ellmer 2018, theory ch. 6): for every stream whose spec allows it (`covariance.<type>.estimate ~= "none"`), the spectral *shape* of its noise is estimated per pool group from Hutchinson-probed redundancies, one *level* per stream, and (with `ArcFactors`) one factor per arc; prior groups with `estimate="scale"` get their sigma by Helmert / Koch–Kusche in the same passes. The v4 words map as: `"ellmer"` → `Estimate="noise"`; `"vce"` → `Estimate="noise", Pool="stream", ArcFactors=false`; `"pooled"` → `Estimate="noise", Pool="all", ArcFactors=false`.

`Pool` — `"all"` (default) | `"obsType"` | `"stream"`. Which streams *share* one estimated spectral shape: all streams of the run; the streams of one observation type across systems (both satellites' KBR, say); or every (system, type) stream on its own. Every stream keeps its own level regardless. Fewer groups = more arcs per shape = a steadier estimate; more groups = more physics.

`ArcFactors` — logical, default `true`. Per-arc variance factors (theory 6.5.4): every arc of a stream gets a keyed factor on top of the stream's level, normalised to unit mean over the run. `false` = one level per stream (the Helmert factor of its arcs). Turn off when arcs are few or short.

`Robust` — `"none"` (default) | `"huber"` | `"igg3"` | `"bisquare"`. M-estimation on the same loop, every data kind, in two deliberately different conventions (chapter 7, §7.8). With `Estimate="none"` it is the stand-alone IRLS (`adjustRobust`): the influence function re-weights the law-whitened *innovations* (Yang) with a MAD scale each pass, `RobustOpts.MaxIter` / `Tol` are its budget, the final factors are `R.history(end).factor`. With `Estimate="noise"` (since 6.1) the robust pass runs *inside* the joint estimator and weights *observations* (Koch): the raw residuals, descaled by each epoch's declared accuracy, one factor per epoch, folded into `Cep` before the temporal law; `MaxPasses` is the budget, the factors are `R.noise.robust.factor`. Prefer `"igg3"` or `"bisquare"` for gross blunders — Huber keeps c/|u| of their weight.

4.2 The loop

`MaxPasses` — integer ≥ 1, default 10. The pass budget of the variance-component loop and of Gauss–Newton. **IRLS has its own**, `RobustOpts.MaxIter` (30), as it has its own tolerance — `engineArgs` hands `adjustRobust` that one and never `MaxPasses` (R6 WP-101). Exhausting the budget returns the last pass with `R.stopReason = "maxPasses"` and one warning.

`Tolerance` — positive, default 1e-2. The stop rule of the variance-component loop: max $|\alpha^2 - 1|$ over frequencies and arcs below it; for Gauss–Newton the relative step in units of the formal error; IRLS uses `RobustOpts.Tol`.

`Probes` — integer ≥ 1, default 100 (Ellmer's value). Hutchinson probes for the per-lag redundancies of the joint estimator. There is **no exact route** in that estimator, so 0 is refused at assignment (R6 WP-101; the `'TraceMethod','exact'` route exists only on the engine-tier `lsqvce.engine.redundancy`, which `fit` does not call).

Seed — `[]` (default) or a double. RNG seed for the probes; recorded in `R.meta.seeds` so a run is reproducible.

4.3 Run control

Backend — `"qr"` (default) | `"normal"`. Square-root (Householder QR, rank refusal by `rank()`'s tolerance) or normal equations (Cholesky). Same numbers to rounding; QR is the safer default on ill-conditioned common blocks.

CacheDays — positive, default 1. Built days kept resident per system. A two-stage pass builds every day twice (accumulate + sweep) and a VCE run 2n+1 times; `Inf` keeps the calendar and builds each day once per run — your memory decision (a built type-day is m × nc doubles, ≈190 MB at degree 60 for a 6-h three-axis piece).

Verbose — logical, 0 or 1, default `false`. Screen messages: 0 silent, 1 progress. This surface is a FLAG: `Options.engineArgs` hands every driver `logical(Verbose)`, so level 2 has never been reachable through `fit` — it is an engine-tier setting of `lsqvce.util.verbosity` (R6 WP-101). A 2 is still accepted here and behaves as 1. (Level 3 was merged into 2 in 2026-07; `lsqvce.util.verbosity` refuses it, and since v5.3 so does `Options` itself, by name.)

OnPass — `[]` (default) or `@(ctx)`. Your inspection hook, called at the end of **every pass** of every iterative run with one struct: `driver`, `pass`, `x`, `Cxx`, `sigma0`, `s0sq`; the loop state (noise: `C`, `Cgroups`, `sigma2`, `arcKeys`, `streams`, `sigma`, `white`, `whitePval`, `maxDev`, `skips`; IRLS: `factor`, `keys`, `scale`, `dx`; Gauss–Newton: `vPv`, `dxRel`, `lambda`); `acc` — the **live accumulator** (call `lsqvce.engine.residuals(ctx.acc, …)` or any engine function inside the hook); `data` — your declaration as passed to `fit`; `processingOptions{k}` — system k's, as declared. Plot, `drawnow`, save wherever you like. Return nothing to continue, `false` to stop after the pass (`R.stopReason = "userStop"`, the last pass returned with its covariance). A throwing hook is not swallowed: it arrives as `lsqvce:callUser:threw` with your error as the cause. This replaces the v4 `plotPostSolve` / `plotPostModel` hooks and `ShowSeconds` / `ExportDir`.

4.4 The groups

A group is a struct; assign a whole (partial) struct or one field. Partial structs are merged onto the defaults; an unknown field is refused (`lsqvce:Options:badGroupField`).

4.4.1 `Noise` — the estimated stochastic model

field	default	meaning
`Grid`	60	frequencies (lags) of the estimated shape; an integer ≥ 2, and ≥ the longest arc for an exact law. `"auto"` was documented until v5.3 and never implemented; it is refused now (R6 WP-101)
`Whitener`	`"toeplitz"`	how a PSD law whitens an arc: exact Toeplitz Cholesky, or an AR / ARMA filter fitted to it (`"ar"`, `"arma"`: O(m·p) instead of O(m²))
`MeanType`	`"robust"`	the normalising mean of the arc factors: trimmed (`"robust"`) or plain
`Smooth`	0	moving-average half-width on the estimated PSD (0 = none)

field	default	meaning
`FreezeRedundancy`	1	a frequency whose redundancy is below this holds its value instead of being updated
`SkipThreshold`	0.2	the skip *rate* (arcs the sweep could not process / arcs of the stream) above which the pass is refused rather than silently thinned

4.4.2 `RobustOpts` — IRLS

field	default	meaning
`Tuning`	`[]`	the influence function's tuning constant (its textbook default when empty: Huber 1.345, IGG3 (1.5, 2.5), bisquare 4.685)
`MaxIter`	30	IRLS passes
`Tol`	1e-8	relative change of x that ends the iteration

4.4.3 `Diagnostics` — the per-observation tests (`R.diag`, `R.report`)

field	default	meaning
`Alpha`	1e-3	significance level of the outlier test
`Bonferroni`	`true`	correct α for the number of tests
`Beta`	0.2	power (1 − β) for the minimal detectable bias
`RMin`	0.01	redundancy number below which an observation is "untestable"
`RWeak`	0.1	below which it is "weakly controlled"
`Test`	`"auto"`	`"pope"` (σ_0 estimated) / `"baarda"` (σ_0 known) / `"mad"` (robust scale) / auto by the a-priori backstop
`RobustCheck`	`true`	run the MAD-scale backstop that flags a contaminated fit
`Store`	`"all"`	keep per-observation arrays for all / `"flagged"` / `"none"`
`NWorst`	1000	how many worst observations `R.report` keeps

4.4.4 `Parallel`

field	default	meaning
`Mode`	`"serial"`	`"parfor"` or `"spmd"` with the Parallel Computing Toolbox; serial fallback with one warning without it
`DistributeDays`	`false`	the day loop of the accumulation over workers (bitwise-identical fold order)
`Threshold`	8	days below which distribution is not attempted

4.4.5 `GaussNewton` — problems with `data.x0`

field	default	meaning
`TolF`	1e-10	relative change of the objective that ends the iteration
`Lambda0`	1e-6	initial Levenberg–Marquardt damping; a rejected step multiplies it

4.5 Retired knobs

Regularisation is no longer an option (`Reg`, `RegLambda`, `RegMatrix`, `DatumB`, `Sigma0`, `Sparse` are gone): declare `data.priors` / `data.systems(k).priors` (chapter 6). The full v4 → v5 map is chapter 12.

5 The user tier: fit, Options, Results, report, layout

The help blocks of the five user-tier units, verbatim (`help lsqvce.<name>` shows the same text; `doc lsqvce.<name>` the same page). This chapter is generated from the code at build time.

5.1 lsqvce.fit

LSQVCE.FIT Fit a least-squares adjustment (optionally with VCE) to declared data.

```
LSQVCE.FIT  Fit a least-squares adjustment (optionally with VCE) to declared data.

  R = lsqvce.fit(data)                        every default (one GLS solve)
  R = lsqvce.fit(data, opts)                  a lsqvce.Options object
  R = lsqvce.fit(data, Name, Value, ...)      the Options properties, straight on fit
  R = lsqvce.fit(data, opts, Name, Value)     both: the pairs override opts

  THE ONE VERB. You describe the problem with a plain struct `data`
  (manual, chapter 3), say what to do with lsqvce.Options (chapter 4), and
  get a lsqvce.Results back (chapter 5):

      R = lsqvce.fit(data, Estimate="noise", Pool="obsType", OnPass=@myInspect);
      R.report;  R.plot("psd");  [C, L] = R.toTable(data);

  THREE DATA KINDS, told apart by the fields of `data`, all on ONE accumulator:
    (A) streaming   data.commonParams, data.days, data.systems(k) with
```

```
                buildDay / covariance / localParams / priors /
                processingOptions and the optional arc schedule `arcs`
                (v6: buildDay returns one CELL per arc -- A, B, l, t,
                Cep as 1 x nArc cells -- and the cells are the schedule
                unless `arcs` declares one); optional data.priors,
                data.paramColumns
  (B) in memory data.A (m x u), data.l (m x 1), optional data.p (m x 1
                weights); optional data.priors. A FULL m x m SPD weight
                matrix is kind C's: return it from batchFcn (R6 WP-101)
  (C) batched   data.batchFcn @(k) -> [A, l, p] (or @(k, x) -> [J, w, p] with
                data.x0 for Gauss-Newton), data.numUnknowns, data.numBatches;
                optional data.priors
WHAT RUNS (opts.Estimate / opts.Robust / data.x0):
  Estimate "none"    one generalised solve under the declared laws (adjustBlock)
  Estimate "noise"   the joint variance-component estimator (adjustVCEjoint):
                     one spectral shape per Pool group, one level per stream,
                     per-arc factors when ArcFactors; prior groups with
                     estimate "scale" get their sigma by Helmert / Koch-Kusche
  Robust "huber" | "igg3" | "bisquare"   IRLS on the same loop: alone it is
                     adjustRobust; with Estimate "noise" the robust pass runs
                     INSIDE the joint loop (R6 WP-105), R.noise.robust carries
                     the final factors
  data.x0            Gauss-Newton with damping (adjustNonlinear)
REGULARISATION is declared, never an option: data.priors(j) on the common
parameters and data.systems(k).priors(l) on the local ones -- one struct per
group, by `group` (a name pattern) or `L` (in the columns lsqvce.layout
announces), x0, sigma, from, name, estimate. A free network's datum is the
prior on its null space.
WATCHING A RUN: Options.OnPass fires after every pass with the pass state,
the live accumulator, `data` and every system's processingOptions.

INPUT
  data   1x1 struct    the declaration (kind A, B or C as above)
  opts   1x1 lsqvce.Options   default lsqvce.Options()
  Name, Value          any property of lsqvce.Options
OUTPUT
  R      1x1 lsqvce.Results   x, sigma0, sol (Cxx, dof, vtv), diag, acc, history,
                              noise (C, Cgroups, stream, model, priors), stopReason,
                              meta (options as resolved, warnings, seeds, version);
                              methods report, summary, convergence, residuals,
                              contribution, compare, plot, toTable, coefficients

ERRORS
  Your declaration is wrong (raised BEFORE any data is read):
    lsqvce:fit:unknownField    a field name the contract does not know
                               (names the field; typos are errors on
                               purpose -- and so are the retired fields:
                               common, numCommon, numLocal,
                               observationTypes, systemId, cache, loader,
                               plotPostSolve, plotPostModel, since v4
                               localsPerArc (declare 'arc' rows in
                               localParams instead), and since v5.3
                               epochCovariance (the day returns Cep))
    lsqvce:fit:noSystems       data.systems missing, not a struct, or empty
    lsqvce:fit:noCommon        data.commonParams missing, or not a 2-D
```

```
                              cell of template rows
  lsqvce:fit:badScope         a scope-token column 1 does not carry:
                              commonParams needs 'common'|'day' (raised
                              here), localParams needs
                              'arc'|'day'|'system' (raised by makeLoader
                              and relabelled)
  lsqvce:fit:missingField     a required system field is absent, or
                              covariance carries no observation type
  lsqvce:fit:noDays           no day list anywhere
  lsqvce:fit:badId            id is not a numeric scalar or single char
  lsqvce:fit:duplicateSystem two systems share an id (explicit or
                              index default)
  lsqvce:fit:builderKind      buildDay is not a function handle
  lsqvce:fit:badArcs          arcs is a string (the v5 'fromDay' is retired:
                              omit arcs -- the cells are the arcs), or a
                              resolver returned something that is not
                              unique positive integers
  lsqvce:fit:<makeLoader id> a makeLoader construction error, relabelled
                              (e.g. paramType, badArcs, localPlan);
                              the original is attached as the cause
Your choice is not available here:
  lsqvce:fit:notYet           Robust ~= "none" on a NONLINEAR batch problem
                              (x0): IRLS inside Gauss-Newton is not written;
                              linearise first. (Robust with Estimate "noise"
                              RUNS since R6 WP-105: the robust pass inside
                              the joint loop)
  lsqvce:fit:modelKind        Estimate = "noise" on kinds B / C, which have
                              no streams to estimate a law for
  lsqvce:batchSystem:weightMatrix  a batch returned a weight argument that
                              is neither m x 1 nor an m x m SPD matrix. A
                              full SPD P IS accepted on kind C and whitened
                              by its Cholesky factor (R5 M5 5b') -- raised
                              by lsqvce.engine.batchSystem
  lsqvce:fit:priorNotActive   (warning, R5-D8) a prior with from > 1 in a
                              single-pass run is not applied
  lsqvce:fit:badPrior         (R5 M5 5b) data.priors is malformed (unknown
                              field, L not p x nc, sigma not a positive scalar,
                              x0 of the wrong length)
  lsqvce:fit:badEstimate      (R5 M3 3d) covariance.<type>.estimate is not
                              "none" | "scale" | "law"
  lsqvce:fit:lawOnFixedStructure  (R5 M4 4b) estimate "law" on a stream whose
                              sigma is per-row (a fixed structure: level only)
  lsqvce:fit:badMatrix        kind B shapes disagree (A m x u, l m x 1,
                              p m x 1)
  lsqvce:fit:missingField     kind C lacks batchFcn / numBatches, or the
                              linear form lacks numUnknowns
  lsqvce:fit:uCount           kind C carries BOTH x0 and numUnknowns --
                              u is stated once (numel(x0))
Your buildDay is wrong (raised WHILE READING, by the engine, ids
unchanged): lsqvce:makeLoader:badDay / dayShape / badArc / badArcId /
buildDay / badDayCep / axisConflict / cepTwice (v5.3: the day's Cep and
axis count), lsqvce:covModel:cepOnVar, lsqvce:accum:arcBadNc / arcBadNl;
lsqvce:makeLoader:noCep is a once-per-run warning (a multi-axis day
without Cep is composed with D = I)
```

```
  See also LSQVCE.OPTIONS, LSQVCE.RESULTS, LSQVCE.LAYOUT, LSQVCE.ENGINE.DECLARATION,
       LSQVCE.ENGINE.MAKELOADER, LSQVCE.ENGINE.ADJUSTBLOCK, LSQVCE.ENGINE.ADJUSTVCEJOINT,
           LSQVCE.ENGINE.ADJUSTROBUST, LSQVCE.ENGINE.ADJUSTNONLINEAR

  Created 2026-08-31 (CEST, UTC+02:00) with assistance from Claude (Anthropic),
  model Claude Fable 5 -- R4 Level L2, WP-56 (decision R4-D4).
  Help rewritten 2026-09-04 for v5.2 (R5 M5 / R5-D8), model Claude Fable 5.1:
  the v5 surface (Estimate, priors, paramColumns, OnPass, layout, toTable).
  Updated 2026-09-09 (CEST, UTC+02:00), R6 WP-105: Robust together with
  Estimate "noise" runs (the robust pass inside the joint loop; the notYet
  refusal stays for x0 problems only). With assistance from Claude
  (Anthropic), model Claude Fable 5.1.
  Updated 2026-09-09 (CEST, UTC+02:00), R6 WP-102 (v6): the day contract is
  ONE CELL PER ARC and the arc schedule is optional. With assistance from
  Claude (Anthropic), model Claude Fable 5.1.
  Updated 2026-09-08 (CEST, UTC+02:00), R6 WP-99: R.noise.model{k}.C is the
  law of the stream's pool group and EMPTY where no law was estimated, instead
  of falling back to another stream's fitted spectrum. With assistance from
  Claude (Anthropic), model Claude Opus 5.
  Updated 2026-09-06 (CEST, UTC+02:00), R6 WP-90: the error information
  travels with the day (D.Cep); epochCovariance retired; the See also on
  the units that exist. With assistance from Claude (Anthropic), model
  Claude Fable 5.1.
```

5.2 lsqvce.Options

LSQVCE.OPTIONS The one settings object for LSQVCE.FIT (v5: 12 knobs + 5 groups).

```
LSQVCE.OPTIONS  The one settings object for LSQVCE.FIT (v5: 12 knobs + 5 groups).

  opts = lsqvce.Options()                        every default
  opts = lsqvce.Options(Name, Value, ...)        override some
  opts.Name = Value                              change one later (validated)
  R = lsqvce.fit(data, Name, Value, ...)         the same names, straight on fit

  WHAT TO ESTIMATE, HOW TO RUN. Everything ABOUT A STREAM (its law, whether
  its law / level / arc factors are estimated) is declared on the stream
  (data.systems(k).covariance.<type>: type, sigma, estimate, arcFactors);
  this object holds only RUN policy. R5 M5 (WP-87, decision R5-D2) cut it
  from 62 flat knobs to the 12 below plus five typed groups; the migration
  from every v4 name is in RELEASE_NOTES.md and HANDOVER §12.10.

  TOP LEVEL
    Estimate   "none" | "noise"   "none": one GLS solve under the declared
                                  laws. "noise": the joint (Ellmer) variance
                                  component estimator on every stream whose
                                  spec allows it (estimate ~= "none"), with
                                  the policy below. Replaces Model: the v4
                                  spellings are Estimate="noise" plus
                                    "ellmer" -> Pool as set, ArcFactors true
                                    "vce"    -> Pool "stream", ArcFactors false
                                    "pooled" -> Pool "all",    ArcFactors false
    Pool       "all" | "obsType" | "stream"   which streams SHARE one estimated
                                  spectral shape (every stream keeps its own
                                  level regardless); default "all"
    ArcFactors logical            per-arc variance factors on (Ellmer 6.5.4),
```

		default true; false = one level per stream. TO READ AN ARC'S STANDARD DEVIATION back, use all three terms (R6 WP-100): declared sigma * sqrt(factor) * R.sigma0 -- the level and the factor split one scale by a convention (eq. 6.5.30's gammaBar) and sigma0 carries the rest, as in any GLS adjustment. MEASURED on a fixture planted at 0.5 / 1.5 / 0.5: 0.517 / 1.499 / 0.507. (R6 F-01, fixed in WP-98: the ratio settles across passes, 9.4 / 8.6 / 8.6 / 8.6 against a planted 9; before it grew 8.4x a pass.)
Robust	"none" | "huber" | "igg3" | "bisquare"	IRLS on the same loop, every kind (details in RobustOpts). With Estimate "noise" the robust pass runs INSIDE the joint loop (R6 WP-105): one residual sweep per pass, the factors in every arc's whitener on the next pass; MaxPasses is the budget, RobustOpts.Tuning the constants (RobustOpts.MaxIter / Tol belong to the stand-alone IRLS only)
MaxPasses	double 10	the pass budget of the variance-component loop and of Gauss-Newton (v4: MaxOuter / JointMaxIter). IRLS has its OWN budget, RobustOpts.MaxIter (R6 WP-101)
Tolerance	double 1e-2	stop when max|alpha2 - 1| over frequencies and arcs is below it (v4: JointTol)
Backend	"qr" | "normal"	square-root vs normal equations
Probes	double 100	Hutchinson probes for the per-lag redundancies; an integer >= 1 (Ellmer's value is 100). The joint estimator has no exact route: 0 is refused (R6 WP-101)
Seed	[] | double	RNG seed for the probes (recorded)
CacheDays	double 1	built days kept resident per system; Inf = the whole calendar (one build per run)
Verbose	logical | 0 | 1	screen messages on this surface: 0 silent, 1 progress. THE VALUE IS PASSED AS A FLAG: engineArgs hands every driver logical(Verbose), so level 2 has never been reachable through fit -- it is an engine-tier setting of lsqvce.util.verbosity (R6 WP-101; 2 is still accepted here and behaves as 1)
OnPass	[] | @(ctx)	YOUR inspection hook, called at the end of EVERY pass of every iterative run (the variance-component loop, IRLS, Gauss-Newton) with ONE ctx struct: .driver .pass .x .Cxx .sigma0, the loop state (noise: .C .Cgroups .sigma2 .arcKeys .streams .sigma .white .whitePval .maxDev .skips; IRLS: .factor .keys .scale .dx; Gauss-Newton: .vPv .dxRel .lambda), .acc (the LIVE accumulator: call lsqvce.engine.residuals(ctx.acc, ...) or any engine function inside the hook), .data (your declaration as passed to fit) and

```
                                .processingOptions (per system, as declared).
                                Plot, drawnow, save wherever you like; return
                                nothing to continue, false to stop after this
                                pass (stopReason "userStop"). A throwing hook
                                is NOT swallowed (lsqvce:callUser:threw with
                                your error as cause). Replaces v4's
                                plotPostModel / plotPostSolve seams.

GROUPS (structs; set fields one at a time or give a whole struct)
  Noise        Grid 60             lags/frequencies of the estimated shape;
                                   an integer >= 2, at least the longest arc
                                   for an exact law ("auto" is refused)
               Whitener "toeplitz"|"ar"|"arma"  how a PSD law whitens (v4 PsdMode)
               MeanType "robust"|"plain"        arc-factor normalisation
               Smooth 0                         moving average on the PSD
               FreezeRedundancy 1               components below it hold
               SkipThreshold 0.2                skip RATE that refuses the pass
  RobustOpts   Tuning [], MaxIter 30, Tol 1e-8
  Diagnostics  Alpha 1e-3, Bonferroni true, Beta 0.2, RMin 0.01, RWeak 0.1,
               Test "auto"|"pope"|"baarda"|"mad", RobustCheck true,
               Store "all"|"flagged"|"none", NWorst 1000
  Parallel     Mode "serial"|"parfor"|"spmd", DistributeDays false, Threshold 8
  GaussNewton  TolF 1e-10, Lambda0 1e-6     (x0 problems; MaxPasses / Tolerance
               are the budget and the step tolerance)

REGULARISATION is not an option: it is DECLARED on the data, as
pseudo-observations in the user's own columns -- data.priors(k) on the
common parameters (L in the columns of A) and data.systems(k).priorDay on
the local ones (L in the columns of B), each with x0, sigma, from, group
(R5-D8). The v4 Reg / RegLambda / RegMatrix / DatumB / Sigma0 / Sparse
knobs are gone (HANDOVER §12.10).

METHODS
  engineArgs(target)   the Name-Value list a driver takes (validated)
  toStruct()           every public property as a struct (provenance)
  disp()               grouped display, engine defaults marked *

ERRORS
  MATLAB:validators:*          a typed property receives a value outside
                               its type / set / range (at assignment)
  lsqvce:Options:badGrid       Noise.Grid is not an integer >= 2 ("auto"
                               was documented until v5.3, never implemented)
  lsqvce:Options:badVerbose    Verbose is neither logical nor an integer 0..2
  lsqvce:Options:badOnPass     OnPass is neither [] nor a function handle
  lsqvce:Options:badGroupField a group struct carries an unknown field
  lsqvce:Options:badTarget     engineArgs target is unknown

EXAMPLE
  opts = lsqvce.Options(Estimate="noise", Pool="obsType");
  opts.Noise.Grid = 2160;  opts.Parallel.Mode = "parfor";
  R = lsqvce.fit(data, opts);
  % Robust is NOT in that line on purpose: Robust ~= "none" together with
  % Estimate = "noise" is refused by fit (lsqvce:fit:notYet). This example
  % carried Robust="huber" until v5.3 and could not run (R6 WP-101).
```

```
  See also LSQVCE.FIT, LSQVCE.RESULTS, the Manual, chapter 3, RELEASE_NOTES.md

  Updated 2026-09-09 (CEST, UTC+02:00), R6 WP-105: Robust is forwarded to
  the joint estimator as its 'Influence' (with RobustOpts.Tuning). With
  assistance from Claude (Anthropic), model Claude Fable 5.1.
  Updated 2026-09-09 (CEST, UTC+02:00), R6 WP-104: the two remaining
  "auto" claims about Noise.Grid corrected -- WP-101 retired the value in
  the code and in the Manual but left it standing on THIS page, which is
  what `doc lsqvce.Options` opens. With assistance from Claude (Anthropic),
  model Claude Fable 5.1.
  Updated 2026-09-08 (CEST, UTC+02:00), R6 WP-100: ArcFactors says how to
  read an arc's sigma back -- all three terms, sigma0 included. With
  assistance from Claude (Anthropic), model Claude Opus 5.
  Updated 2026-09-08 (CEST, UTC+02:00), R6 WP-98: ArcFactors says what is
  identifiable (the ratio) now that F-01 is fixed. With assistance from
  Claude (Anthropic), model Claude Opus 5.
  Created 2026-09-03 (CEST, UTC+02:00) with assistance from Claude
  (Anthropic), model Claude Fable 5.1 -- R5 Level M5, WP-87 (v5 interface).
```

5.3 lsqvce.Results

LSQVCE.RESULTS The outcome of a recovery: solution, diagnostics, provenance.

```
LSQVCE.RESULTS  The outcome of a recovery: solution, diagnostics, provenance.
  R = lsqvce.Results(sol, D, acc, diagSol, batchFcn, nBatch, systems)
  R = lsqvce.Results(sol, D, acc, diagSol, batchFcn, nBatch, systems, meta)

  Returned by every driver LSQVCE.FIT dispatches to -- LSQVCE.ENGINE.ADJUSTBLOCK,
  ADJUSTVCEJOINT, ADJUSTROBUST, ADJUSTNONLINEAR. It gathers
  everything a user needs to read and judge an adjustment in one object, and
  keeps the lower layers reachable for experts. You rarely construct it
  yourself; you receive it from adjustBlock and call R.report().

  PROPERTIES (read-only)
    x          nc x 1   double   Common-parameter estimate (the field).
    sigma0     1x1      double   A-posteriori variance factor (global).
    stopReason 1x1      string   Why the loop stopped (R5 M1): "single" |
                                 "converged" | "maxPasses". Convergence
                                 failure is a RESULT: the last pass is
                                 returned with its covariance and one warning.
    converged  1x1      logical  Did the pipeline converge? (true for a single
                                 solve; set by the iterative engines.)
    diag       1x1      struct   Per-observation diagnostics (LSQVCE.ENGINE.DIAGNOSE):
                                 redundancy, leverage, w-tests, MDB, ...
    history    1xI      struct   Per-iteration record (empty for a single
                                 solve; filled by LSQVCE.ENGINE.ADJUSTVCEJOINT:
                                 .maxDev .Shat .streams .sigma .white
                                 .whitePval per pass, .skips, .nIter, and
                                 under Robust the per-pass .robustScale (one
                                 column per stream) .nDown .nZero -- R6 WP-105;
                                 by LSQVCE.ENGINE.ADJUSTROBUST: .scale .dx
                                 .nDown .nZero and, on the last pass, .factor
                                 .keys).
    noise      1x1      struct   Fitted noise models per stream (empty until
                                 the variance-component loop populates it):
                                 .stream (keys) .model{k} (.sigma the level,
```

```
                              .C the law) .Cgroups (one C per pool group)
                              .priors .priorHistory, and .robust
                              (R6 WP-105: .influence, .scale per stream,
                              .streams, .factor ONE PER EPOCH in sweep
                              order, .keys; empty fields under "none").
  sol        1x1      struct   Raw output of acc.solve() (.x, .sigma0, ...).
  acc        1x1      lsqvce.engine.BlockAccumulator   The accumulator (drop here
                              to re-solve, inspect blocks, get residuals).
  source     1x1      struct   Bridged solution from LSQVCE.ENGINE.DIAGNOSEACCUM.
  batchFcn   1x1      function_handle   Re-reader used by LSQVCE.ENGINE.DIAGNOSE.
  nBatch     1x1      double   Batch count from the bridge.
  systems    1xM      struct   The resolved system specs (sysID/nl/dayList).
  meta       1x1      struct   Run provenance (WP-44). Field list is
                              ADD-ONLY across versions, never renamed:
                              .toolboxVersion 1xK char  from Contents.m,
                                              the ONE version source
                              .matlabRelease  1xK char  version('-release')
                              .createdUTC     1xK char  ISO 8601, trailing Z
                              .seeds          1x1 struct every RNG seed the
                                              run resolved (struct() when
                                              none; a resolved-but-unset
                                              seed is stored as [])
                              .options        1x1 struct the RESOLVED
                                              options the run actually used
                              A bare construction (7 arguments) gets a
                              meta with empty seeds/options -- the field
                              is never missing.

METHODS
  V = R.report(Name,Value,...)   Print/return the level-0..4 verdict
                                 (LSQVCE.REPORT); options pass straight through.
  r = R.redundancy()             Total redundancy sum(r_i) = n - u.
  res = R.residuals(Name,Value)  Whitened/raw residuals (acc.residuals).
  con = R.contribution(...)      Per-group information share.
  T = R.convergence(Name,Value)  Per-ITERATION trajectory: when the loop
                                 settled (needs .history).
  F = R.compare(Rred)            Nested-model F test against a reduced fit (R5 M7).
  fh = R.plot(kind)             "residuals" | "contribution" | "psd" | "convergence"
                                 (R5 M5: one figure per question, nothing exported).
  [C, L, map] = R.toTable(data)  The expanded parameter tables with estimate and
                                 standard deviation written in (R5-D8).
  T = R.summary()                Per-STREAM final state: sigma, AR order,
                                 whiteness (needs .history).
  T = R.coefficients(Alpha=a)    The estimates as a table with formal
                                 errors and (1-a) confidence bounds
                                 (R4 WP-57; default a = 0.05).
  disp(R)                        One-screen summary.

THE TWO ITERATION READERS ARE COMPLEMENTARY. convergence() reduces across
STREAMS (one row per pass); summary() reduces across PASSES (one row per
stream). A converged loop can still hide a single stream that never
whitened -- the per-pass view reports the worst p-value without naming it.

EXAMPLE
  R = lsqvce.engine.adjustBlock(systems, days);
```

```
  R.report('Level', 2);
  if ~R.converged, lsqvce.util.log.warn('did not converge'); end
  x = R.x;   s0 = R.sigma0;

ERRORS
  lsqvce:Results:noVPv          compare() on a Results without a finite v'Pv
  lsqvce:Results:noHistory      an iteration reader was called on a
                                single-solve Results (adjustBlock does not
                                iterate); use the result of adjustVCE
  lsqvce:Results:noCovariance   coefficients() on a Results whose sol
                                carries no Cxx
  lsqvce:Results:noAccumulator  residuals()/contribution() on a Results
                                without an accumulator (fit kinds B / C)

ACCURACY
  Results adds no numerics: x and sigma0 are the engine's (sigma0 is the
  A-POSTERIORI std of unit weight; sol.Cxx the covariance scaled by it --
  the streaming contract every producer must establish, R4 §11.8 L4 r1),
  report() is lsqvce.report on .diag, coefficients() is arithmetic on x
  and diag(Cxx) with z(0.05) = 1.959963984540054 in closed form.

See also LSQVCE.FIT, LSQVCE.ENGINE.ADJUSTBLOCK, LSQVCE.ENGINE.ADJUSTVCEJOINT, LSQVCE.REPORT,
         LSQVCE.ENGINE.DIAGNOSE, LSQVCE.ENGINE.BLOCKACCUMULATOR

Updated 2026-09-08 (CEST, UTC+02:00), R6 WP-99: R.report() forwards the
loop state, so the verdict says whether the stochastic model settled.
With assistance from Claude (Anthropic), model Claude Opus 5.
Updated 2026-09-01 (CEST, UTC+02:00), R4 WP-61 (Level L3): ACCURACY block
(the sigma0 / Cxx contract stated once). With assistance from Claude
(Anthropic), model Claude Fable 5.
Updated 2026-08-31 (CEST, UTC+02:00), R4 WP-56b (Level L4): residuals()
and contribution() refuse with lsqvce:Results:noAccumulator on a Results
without an accumulator (the in-memory / batched kinds of fit). With
assistance from Claude (Anthropic), model Claude Fable 5.
Updated 2026-08-31 (CEST, UTC+02:00), R4 WP-57 (Level L2): coefficients()
-- estimates, formal errors and confidence bounds as ONE table (the
fitlm Coefficients/coefCI pair folded into one method; z from erfcinv,
no Statistics Toolbox). With assistance from Claude (Anthropic), model
Claude Fable 5.
Updated 2026-08-19 (CEST, UTC+02:00), WP-44 (T-05): run provenance --
immutable .meta composed by the ONE private builder runMeta, constructor
widened to an optional 8th argument, report() forwards the provenance
line. With assistance from Claude (Anthropic), model Claude Fable 5.
Updated 2026-08-19 (CEST, UTC+02:00), WP-39 stage (c) c2 (F-07):
typed `arguments` signatures, canonical UpperCamelCase option names
(legacy casings keep matching -- MATLAB NV semantics; enum VALUES
case-sensitive lowercase). With assistance from Claude (Anthropic),
model Claude Fable 5.
Created 2026-07-16 12:20 (CEST, UTC+02:00) with assistance from
Claude (Anthropic), model Claude Opus 4.8.
```

5.4 lsqvce.report

LSQVCE.REPORT Judge a least-squares adjustment and say so in words.

```
LSQVCE.REPORT  Judge a least-squares adjustment and say so in words.
```

```
V = LSQVCE.REPORT(D) takes the struct from lsqvce.engine.diagnose and prints a
verdict. It returns a struct of machine-readable findings so the same logic
can drive an automated pipeline.

The report is ordered so that a failure at one level invalidates the levels
below it. It refuses to report parameter precision for an adjustment whose
integrity checks failed, because a sigma computed from a broken system is
not a conservative estimate -- it is a fabrication.

LEVELS
  0  INTEGRITY   sum(r) == n-u, vPv >= 0, vPv consistent between passes.
                 These test the CODE, not the data. Failure here means
                 every number below is meaningless.
  1  GLOBAL      variance factor vs chi-square.
  2  LOCALISE    outliers (data snooping), with Bonferroni.
  3  RELIABILITY redundancy numbers, MDB, external reliability.
                 This is the level that says which observations you are
                 NOT ABLE TO CHECK.
  4  PARAMETERS  correlations, conditioning, precision.

NAME-VALUE
  'verbose'  true  print the report (default true)
  'width'    78    line width
  'Provenance' struct()  a run-provenance struct (LSQVCE.RESULTS .meta,
                   WP-44); when non-empty, ONE level-0 NOTE line
                   (code 'PROVENANCE') states toolbox version, MATLAB
                   release, and creation time. R.report() forwards it
                   automatically; a bare lsqvce.report(D) prints none.
  'Iteration'  struct()  the loop state of an ITERATIVE run (R6 WP-99):
                   .converged, .stopReason, .nIter. When non-empty and the
                   run was not a single solve, ONE level-1 line says
                   whether the stochastic model settled -- 'NOISE_SETTLED'
                   (NOTE) or 'NOISE_UNSETTLED' (WARN). Every number from
                   level 1 down rests on those weights, so a model that was
                   still moving qualifies all of them. R.report() forwards
                   it automatically.

OUTPUT V
  V.verdict   'PASS' | 'PASS WITH WARNINGS' | 'SUSPECT' | 'INVALID'
  V.findings  cell array of struct(level, severity, code, msg)
  V.nError, V.nWarn, V.nNote

INPUT
  D         1x1 struct  diagnostics from LSQVCE.ENGINE.DIAGNOSE
  varargin  Name/Value: 'verbose', 'width', 'alpha', 'rWeak', ...

EXAMPLE
    data = struct('batchFcn', src, 'numUnknowns', u, 'numBatches', 4);
    R = lsqvce.fit(data, Verbose=false);
    V = lsqvce.report(R.diag);         % prints the five levels (= R.report)

    % V.verdict    'PASS' | 'PASS WITH WARNINGS' | 'SUSPECT' | 'INVALID'
    % V.findings   machine-readable: {level, severity, code, msg}
```

```
    % Drive a pipeline off the codes rather than the text:
    codes = cellfun(@(f) f.code, V.findings, 'UniformOutput', false);
    if ismember('SNOOP_CONTAMINATED', codes)
        error('Pope is blind here -- re-run with test=mad.');
    end

  Updated 2026-08-19 (CEST, UTC+02:00), WP-39 stage (b) b2 (F-07):
  typed `arguments` signature, canonical UpperCamelCase option names
  (legacy casings keep matching -- MATLAB NV semantics; enum VALUES
  case-sensitive lowercase). With assistance from Claude (Anthropic),
  model Claude Fable 5.
  See also LSQVCE.ENGINE.DIAGNOSE, LSQVCE.ENGINE.SYSTEMATIC.

  Updated 2026-09-08 (CEST, UTC+02:00), R6 WP-99: the new 'Iteration'
  option puts ONE level-1 line in the verdict saying whether the stochastic
  model settled -- a report that stays silent about weights that were still
  moving is a report that misleads. With assistance from Claude (Anthropic),
  model Claude Opus 5.
  Generated with assistance from Claude (Anthropic), Claude Opus 4.8,
  2026-07-11.
```

5.5 lsqvce.layout

LSQVCE.LAYOUT The column layout fit will use – so you can declare priors in it.

```
LSQVCE.LAYOUT  The column layout fit will use -- so you can declare priors in it.
  lay = lsqvce.layout(data)

  The toolbox derives from your parameter tables (scope tokens, calendar,
  arcs, validity) which column of A and of every system's B belongs to
  which instantiated parameter. This function announces that layout BEFORE
  fit runs, so that data.priors(j).L (columns of A) and
  data.systems(k).priors(l).L (columns of B) can be written in it -- the
  only knowledge the user's regularisation and the toolbox share (R5-D8).

  INPUT   data  1x1 struct  a kind-A declaration (the Manual, chapter 3)
  OUTPUT  lay   1x1 struct
    .nc            1x1        columns of A
   .common      nc x 1 struct (.row .day)    the template row and (for 'day' rows) the day of every col
    .systems(k)    1x1 struct per system:
       .nl         1x1        columns of B
     .local    nl x 1 struct (.row .day .arc) the template row / day / arc of every column of B
      .col      @(row, day, arc) -> column    lookup: the column of template row `row` on
                                          `day` (and `arc` for 'arc' rows; [] otherwise);
                                              0 when that instance does not exist
    .colCommon    @(row, day) -> column        the same for A ([] day for 'common' rows)

  EXAMPLE  (a Tikhonov prior on local row 7 of system 1, on every day)
    lay = lsqvce.layout(data);
    nl  = lay.systems(1).nl;  rows = zeros(0, nl);
    for d = data.days
      c = lay.systems(1).col(7, d, []);  r = zeros(1, nl);  r(c) = 1;  rows(end+1, :) = r;
    end
   data.systems(1).priors = struct('L', rows, 'x0', 0, 'sigma', 0.01, 'name', 'xaccscale', 'estimate

  See also LSQVCE.FIT, LSQVCE.RESULTS/TOTABLE, the Manual, chapter 3
```

Created 2026-09-03 (CEST, UTC+02:00) with assistance from Claude (Anthropic), model Claude Fable 5.1 -- R5-D8 round 5.

6 Regularisation: priors and groups

6.1 The rule

Parameters and their columns are yours — the tables, `buildDay`, plots, `toTable`. *Regularisation is the toolbox's* — the mechanism, the redundancy, the activation pass, the estimation of its strength. The interface between them is an operator in your columns, announced by `lsqvce.layout`, checked by width.

6.2 What a prior is

Tikhonov regularisation of a parameter is an additional observation equation, "parameter j is observed to be x_0 with standard deviation σ":

$$L \cdot x = x_0 + v, \quad \text{weight } 1/\sigma^2$$

where L is the coefficient matrix of the pseudo-observations (one row per pseudo-observation) exactly as A is the coefficient matrix of the observations. The toolbox therefore treats a prior as data: it enters the normal equations, it has a redundancy (how much of it the data overrule), and estimating its σ is variance-component estimation on that group (Koch–Kusche).

6.3 The declaration — one mechanism for common and local parameters

```
data.priors(j)            = struct(...)    % on the common parameters, L in the columns of A
data.systems(k).priors(l) = struct(...)    % on system k's local parameters, L in the columns of
```

One struct per **group**, with the fields

field	meaning
`group`	a regular expression matched against the *name column* of the table (column 2; `data.paramColumns.name` to say otherwise). The toolbox builds L: one row per matched template row per instance (per day for a `'day'` row, per arc for an `'arc'` row, once for `'system'/'common'`).
`L`	the explicit operator (q × nc or q × nl) in the columns that `lsqvce.layout(data)` announces. Give `group` **or** `L`, never both.
`x0`	the a-priori values (q × 1, or a scalar; default 0)
`sigma`	ONE standard deviation for the rows of the group. Relative weights inside a group go into L (divide a row by its own relative σ); the struct's σ is the common factor.
`from`	the pass at which the group becomes active (default 1). In a single solve a `from > 1` group is reported (`lsqvce:fit:priorNotActive`) and skipped.
`name`	the label (default the pattern); local groups become the stream `prior_<name>`.

field	meaning
`estimate`	`"none"` (default; σ fixed) or `"scale"`: σ is estimated every pass of `Estimate="noise"` by the Helmert rule $\sigma^2 \leftarrow \sigma^2 \cdot \Omega/r$ — the fixed point $\Omega = r$ ("the prior's weighted residual sum equals its redundancy"), so $\lambda = s_0^2/\sigma^2$ is the regularisation the data support.

Several rows make σ estimable. A group with one row has redundancy ≤ 1 and cannot estimate its own σ (it is left unchanged and flagged in `R.noise.priorHistory`). The x/y/z accelerometer scales of both satellites over a month are one group with 180 rows.

Local rows live in one day. A row of a local group has its non-zero columns inside one day's local block (or the system block); coupling two days is done through the common parameters. Different days of the same parameter are different rows — that is how the estimate pools information across the calendar.

6.4 How they are treated

Common groups are stacked into one prior on the common block, folded into the solve and appended to the diagnostic sweep (they carry a redundancy). Local groups ride as their own observation stream `prior_<name>` of the system — pseudo-observation arcs with $A = 0$, $B = L/\sigma$, $l = x_0/\sigma$ and a unit law — so accumulation, elimination, the sweeps, `residuals()` and the redundancy count see them as data, and under `Estimate="noise"` the group's σ is the stream's level: identical behaviour to the common groups, no second estimator. `R.noise.priors`, `R.noise.priorHistory` (common) and `R.noise.model{k}.sigma` for stream `satKey|prior_<name>` (local) report the result.

6.5 The datum of a free network

Minimum constraints $B^Tx = 0$ are the group `L = B', x0 = 0`: when B spans the null space the solution is the exact minimum-constraint one for **any** weight (the data's range and the null space are orthogonal). Without it the accumulator refuses a rank-deficient system by name (`lsqvce:qr:singularR`); it never guesses.

6.6 The layout

```
lay = lsqvce.layout(data);
lay.nc, lay.common(j)                    % the template row (and day) of every column of A
lay.systems(k).nl, lay.systems(k).local(c)   % (row, day, arc) of every column of B
c = lay.systems(k).col(row, day, arc); c = lay.colCommon(row, day);   % lookups (0 = not instanti
```

6.7 Your table's column 11

`{'Tikhonov', fromPass, sigma[, label]}` on a row means exactly one of the structs above; `template/gravfire_data.m` (STEP 6/7) emits them: rows sharing a label become one group in the `group` form with `estimate="scale"`, rows without a label one fixed-σ group each.

7 Stochastic models and their estimation

This chapter is the reference for the covariance specification: what you declare, what travels with the data, how the two are composed, what the estimator does with them, and how to read the result back. Chapter 3 lists the fields as a contract; this chapter says what they mean and how to choose them.

7.1 7.1 The division of labour — three facts, three places

The noise of one observation stream is three separate facts, and each is stated in exactly one place:

fact	where it is stated	who owns it
the **temporal law** R_t — how the errors of a stream correlate down the epochs	`covariance.<stream>.type` and its parameters	you declare a starting point; the estimator refines it
the **level** of that law	`covariance.<stream>.sigma` (always a standard deviation)	you declare it; the estimator scales it
the **per-epoch accuracy** D — what the preprocessing knows about each observation	`D.Cep`, returned by `buildDay` with the day (chapter 3, §7)	you, always; it is never estimated

The toolbox composes them per arc as **C = D (R_t ⊗ I_d) D**: every epoch is de-scaled by D^-1/2, then the temporal whitener runs down the epochs of each axis. Batches are overlapped by the filter memory automatically, so cutting an arc into batches never changes the result.

`Cep` is **relative to** `sigma`. A `diag` law of level s with a per-epoch sigma c whitens as `l ./ (s*c)`. If `Cep` already carries the absolute accuracy — a PPP covariance in m², for instance — declare `sigma = 1`, or the accuracy is counted twice.

7.2 7.2 `Cep` — the three shapes, and the asymmetry that bites

`D.Cep` is a **cell like `A`**, one entry per arc (v6). Each entry may be:

shape	meaning	what the numbers are
nEpoch × 1	a scalar stream with per-epoch accuracy	**standard deviations**
nEpoch × d	a multi-axis stream, axes uncorrelated within an epoch	**standard deviations** per axis
nEpoch × d × d	a multi-axis stream, full per-epoch covariance	**covariance matrices** (the toolbox takes their Cholesky factor)

The vector forms are sigmas and the matrix form is a covariance. That asymmetry is the commonest way to be wrong by a square: check one epoch by hand before a campaign.

Rules:

- the axis count is `d = size(Cep, 2)`, derived, and the same for the whole day; a `nAxis` given as well must agree (`axisConflict`);
- a `Cep` on the covariance spec *and* on the day is refused (`cepTwice`) — one fact, one authority. The spec field is the engine-tier carrier the loader fills from the day;
- a `var` law takes **no** `Cep` (`cepOnVar`): it carries its own cross-axis structure;
- a multi-axis day **without** `Cep` is composed with D = I and reported once per run (`noCep`), so an omission is distinguishable from a choice;
- rows of a multi-axis stream are **epoch-major** ($x_1\ y_1\ z_1\ x_2\ y_2\ z_2\ \ldots$) and every arc holds whole epochs. The temporal law is placed on the **epoch** grid, `t(1:d:end)`.

7.3 7.3 The temporal laws

type	fields	when to use it	cost per batch
`diag`	`sigma`	white noise — and the honest **starting point** when the colour is unknown: `Estimate="noise"` discovers it	O(b)
`arma`	`ar = [1 a₁ … a_p]`, `ma` (default 1), `sigma` (innovation std)	a known whitening filter	O(b·p)
`psd`	`f` (Hz, one-sided), `S` (unit²/Hz)	noise given as a measured spectrum	O(b·p) as AR, O(b³) as Toeplitz
`acf`	`acf = @(dt, opt)`, `sigma`	correlation as a function of the lag	O(b³)
`var`	`ar` (d×d×p), `sigma` (d×d)	genuine **cross-axis temporal** correlation	O(b·p·d²)
`custom`	`fcn = @(t, opt)`	anything else, dense	O(b³)

`sigma` is a standard deviation, never a variance.

A PSD is best whitened as an AR filter, and that is exact. Fitting an AR(p) to the spectrum and whitening with the FIR filter is not an approximation of the dense Cholesky: the filter *is* the Cholesky inverse factor, and both are lower-triangular square roots of the inverse covariance, unique up to sign, agreeing to ~1e-14 row for row. Set `Noise.Whitener = "ar"` and pay O(b·p) instead of O(b³). Validate the fit once per stream with `lsqvce.engine.checkPSD` (aim for `acvRelErr < 1e-3`; raise `arOrder`, or fall back to `"toeplitz"` if it does not reach that).

7.4 7.4 The two policy fields on a spec

Beside the law, a spec may carry:

field	values	meaning
`estimate`	`"law"` (default), `"scale"`, `"none"`	`"law"`: the spectral shape **and** the level are estimated; `"scale"`: your declared shape is kept, only the level is estimated; `"none"`: the stream is left exactly as declared
`arcFactors`	logical	per-arc variance factors for this stream (Ellmer §6.5.4); default: the `Options.ArcFactors` policy

One refusal to know about: a **per-row `sigma` vector** makes the spec a *fixed structure*. It can carry a level but not a spectral shape, so `estimate = "law"` on it is `lsqvce:fit:lawOnFixedStructure`. A `Cep` does **not** trigger this — per-epoch accuracy together with an estimated temporal law is exactly the supported combination, and the reason the error information travels with the day at all.

`Options.Pool` decides which streams **share one estimated spectral shape**; every stream keeps its own level regardless. `"all"` (the default) pools every stream, `"obsType"` groups by stream name, `"stream"` gives each (system, stream) its own shape. With a single system, `"obsType"` and `"stream"` are the same grouping. Pooling streams of different physics is a mistake worth naming:

pooling positions with range-rate was measured to drive the conditioning to RCOND 1e-50 on the gravity demo.

7.5 7.5 What `Estimate="noise"` does

The joint estimator (theory chapter 6; Ellmer 2018 §6.5.7) treats every PSD frequency and every arc factor as variance components of **one** Helmert VCE and updates them together. Per pass it

1. accumulates and solves under the current laws,
2. sweeps every arc once for its whitened residual and its Hutchinson-probed redundancies at every lag (all lags from one FFT per column),
3. updates the spectral shape per pool group in the PSD domain (positive by construction, resampled to each arc's length), the level per stream, the per-arc factors, and the sigma of any prior group with `estimate = "scale"`,

and stops at `Tolerance` (max |alpha² − 1|) or `MaxPasses`. `Noise.Grid` is the number of lags/frequencies of the estimated shape: at least the longest arc for an exact law. `Probes` is the Hutchinson probe count (100 is Ellmer's value); the joint estimator has no exact route, so `Probes = 0` is refused.

Arcs can be **skipped** for a pass with reason `gap` (no data) or `nonPD` (the arc's covariance is not positive definite — a mis-scaled `Cep` produces exactly this). A skipped arc keeps its previous factor and is listed in the run's skip record; beyond `Noise.SkipThreshold` (0.2) the pass is refused rather than estimated on a fraction of the data.

For multi-axis streams the residuals are de-scaled by D **before** the lag statistics are formed, per axis over epochs. The estimated law is therefore the law of the *normalised* errors, which is what makes a 3-axis stream with a per-epoch 3×3 and a scalar stream with a per-epoch sigma behave identically.

7.6 7.6 Worked example — kinematic orbits with a 3×3 per epoch, KBR with one variance

The GRACE-FO case: two position streams whose preprocessing delivers a full 3×3 covariance per epoch, one range-rate stream with a single scalar variance, all in one system (the range-rate rows depend on both satellites' local parameters), and Ellmer's estimator.

```
% ---- the specs: the TEMPORAL law and its level, nothing else ----------
% positions: Cep carries the absolute accuracy -> level 1; white start, so
% the estimator finds the colour; per-arc factors on
cov.posA = struct('type','diag', 'sigma', 1, 'estimate', "law");
cov.posB = struct('type','diag', 'sigma', 1, 'estimate', "law");
% range-rate: the scalar variance IS the level; no Cep on the day
cov.kbr  = struct('type','diag', 'sigma', sqrt(varianceKbr), 'estimate', "law");
s.covariance = cov;

% ---- buildDay: the error information travels with the day ------------
%   D.Cep{k} = nEp_k x 3 x 3     the per-epoch covariance of arc k
%   (rows epoch-major; the kbr stream returns no Cep field at all)

% ---- the run ----------------------------------------------------------
opts = lsqvce.Options( ...
        Estimate   = "noise", ...          % the joint (Ellmer) estimator
        Pool       = "obsType", ...        % one shape per stream, NOT one for all three
        ArcFactors = true, ...             % Ellmer eq. 6.5.4
        MaxPasses  = 7, ...
        Probes     = 100, Seed = 1, ...    % reproducible probes
```

```
        Noise      = struct('Grid', 2160, 'Whitener', "ar"), ...
        CacheDays  = Inf, ...
        Verbose    = 1);
R = lsqvce.fit(data, opts);
```

Why each line: `sigma = 1` on the positions because the 3×3 already carries m²; `Pool = "obsType"` because positions and range-rate differ in units and in spectrum; `Grid = 2160` because that is the length of the longest arc in epochs (6 h at 10 s, 3 h at 5 s); `Whitener = "ar"` because a 2160-epoch arc on the dense Toeplitz path is O(b³).

Before trusting the numbers, run one pass with `Estimate = "none"`. If sigma0 is near 1 the declared weights are consistent. A sigma0 far from 1 says the `Cep` scale is wrong — most often an information matrix used where a covariance was expected — and no amount of estimating a law on top of it will fix that.

7.7 7.7 Reading the result back

```
R.noise.Cgroups          % one covariance function per pool group
R.noise.model{k}.sigma   % the estimated LEVEL of stream k
R.noise.model{k}.C       % its law; EMPTY where no law was estimated
R.noise.stream           % the stream keys, in the order of model{}
R.history(end).white     % Ljung-Box whiteness per stream (K = 20, alpha = 0.05)
R.convergence(); R.plot("psd"); R.plot("convergence");
```

An arc's standard deviation is **all three terms**: declared `sigma` × sqrt(arc factor) × `R.sigma0`. The level and the factor split one scale by a convention (Ellmer eq. 6.5.30's gamma-bar normalisation) and sigma0 carries the rest, as in any GLS adjustment. Measured on a fixture planted at 0.5 / 1.5 / 0.5: 0.517 / 1.499 / 0.507. With `sigma = 1` and a `Cep` carrying the absolute accuracy, that product reads directly as "by how much the preprocessing's covariance was too optimistic on this arc".

The acceptance of the estimator is physical, and its two halves are pinned differently. The planted FIELD is recovered within its formal errors (`demo_gravity` reports z-scores per coefficient; `test_physOracle` asserts them). The planted stream SIGMAS carry no formal error, and what the suite asserts is a RELATIVE bound: the ratios between streams to 25 % (measured 1.18 / 2.10 / 0.51 against a planted 1 / 2 / 0.5).

7.8 7.8 Robustness: what Ellmer does, and what `Robust` adds inside the loop

Three things are worth separating, because Ellmer's own robustness is not per-observation M-estimation and all three of his are available with `Estimate = "noise"` alone:

1. **Gross outlier screening happens in preprocessing**, before the adjustment (Ellmer §6.2, after Klinger 2018: the level-1B data is screened for outliers and periods of reduced quality, and segments shorter than one hour are discarded). Nothing in the adjustment replaces that.
2. **The arc-wise variance factors ARE the down-weighting** at arc granularity: a bad arc receives a large variance factor and loses weight automatically. That is `ArcFactors = true`.
3. **The normalisation of those factors is itself robust**: gamma-bar (eq. 6.5.29) is computed in GROOPS from the central half of the sorted factors, ignoring both tails. That is `Noise.MeanType = "robust"`, the default here.

Per-epoch down-weighting inside the same loop (since 6.1), in the Koch convention. `Robust = "huber" | "igg3" | "bisquare"` together with `Estimate = "noise"` runs the robust pass *inside* the joint estimator. Every pass ends with one residual sweep; the **raw** residuals, descaled by each epoch's own declared accuracy (`Cep`) and reduced to one value per epoch (the RMS over a multi-axis epoch's rows), go through the influence function per stream (`RobustOpts.Tuning` for the constants), and the factors are folded into the per-epoch error information of the next

pass — sigma / sqrt(w), **before** the temporal law, through the one composition C = D (R_t ⊗ I_d) D. Accumulation, the probe redundancies and the component update therefore see **one** re-weighted problem with nothing added. A weight is a statement about the *observation*; the stand-alone IRLS (`Estimate = "none"`) weights the law-whitened *innovation* instead (Yang 1994), and the two are deliberately different: the control measurement showed that an innovation weight of zero neutralises the whitening filter's whole tail around a blunder, so a spectral estimator loses exactly the lag correlations it needs there (a planted colour of 0.20 crawled to 0.054 in 16 passes), while the same blunders declared through `Cep` let the plain estimator recover it. `MaxPasses` is the budget (`RobustOpts.MaxIter` / `Tol` belong to the stand-alone IRLS). Prior streams are never re-weighted: a down-weighted datum is a rank defect, not an outlier. What comes back: `R.noise.robust` (`influence`, `scale` per stream, `factor` per **epoch** in sweep order, `keys`), and `R.history.robustScale` / `nDown` / `nZero` per pass.

Two things the mechanism learned by measurement, both binding on how to use it. **The standardisation is the stream's own MAD scale, never the estimated level**: a global scale mis-standardises every stream whose level differs from the mix, and using the level the whitener just applied makes the weights feed on the level they shape — the clean stream's level spiralled to 0.38 that way. With `v / MAD(v)` per stream the weights are invariant to the level, and the level converges. **Prefer a redescending influence for gross blunders**: Huber is monotone and keeps c/|u| of a blunder's weight, so 2 % of a stream at 25 sigma retain 0.02 · 1.345 · 25 ≈ 0.67 of white variance and dilute a lag-1 colour of 0.20 to about 0.12; `igg3` (zero beyond 3 sigma) and `bisquare` reject them outright and recover the colour.

The level, measured. The obvious expectation — that a robust pass estimates the level of the *retained* observations, E[w u²]/E[w] (0.73 for `igg3`, 0.80 for Huber on Gaussian residuals) — was written down and then **refuted by the oracle**: a clean stream comes back at 1.007. The reason is Ellmer's per-lag redundancy s_n (eq. 6.5.23): it is the model expectation of Omega_n under the *same* weighted whitening, so a down-weighted row shrinks numerator and denominator alike and alpha² = Omega/s stays at 1. This differs from the classical Helmert Omega/(n − u), where a zero-weight row still counts a full redundancy and the level would be biased low. No consistency correction is needed, and none is applied. This composition is Koch's iterated re-weighting inside a variance-component estimation, not something Ellmer (2018) does.

Two further routes remain useful:

- **Diagnose, then decide.** Run `Estimate = "noise"` and read `R.report()` — the w-tests, redundancies and MDBs identify the observations carrying the outliers; remove or repair them in your `buildDay` and re-run.
- **Fold fixed weights into `Cep`.** Take factors from any robust run and let `buildDay` return `Cep` divided by sqrt(factor) on those rows. The weights are then data; the noise estimation runs on the re-weighted stream without the retained-observation bias moving with every pass. The weights must be stored, never recomputed inside `buildDay` (the day builder must be deterministic).

8 Diagnostics and the verdict

Every solve carries per-observation diagnostics in `R.diag` (residuals in the whitened frame `sqrt(p).*v`, redundancy numbers, standardised residuals, the Baarda / Pope test with `Diagnostics.Alpha` and Bonferroni, minimal detectable biases, reliability, the a-priori backstop). `R.report(Level=0..4)` reduces them to one verdict and its evidence; `R.summary()` and `R.convergence()` read the pass record of an iterative run; `R.residuals()` re-reads the arcs for the whitened residuals per stream; `R.contribution()` gives the information share of every stream in the common parameters; `lsqvce.engine.snoop` iterates data snooping with a clean re-solve on the block path; `R.compare(Rred)` is the nested-model F test between two fits. For correlated batches the per-observation quantities live in the whitened frame; only their sums are frame-invariant.

9 Watching a run

`Options.OnPass` (chapter 4) is called after every pass with the pass state, the live accumulator, your declaration and your `processingOptions` — plot, `drawnow`, save, or return `false` to stop. Afterwards `R.plot("residuals" | "contribution" | "psd" | "convergence", Stream=…, Visible=…)` returns one figure per question; nothing is exported by the toolbox. `R.meta.warnings` lists every warning identifier of the run; `R.stopReason` says why the loop stopped (`"single"`, `"converged"`, `"maxPasses"`, `"userStop"`).

```
function go = myInspect(ctx)
    res = lsqvce.engine.residuals(ctx.acc, struct('x', ctx.x, 'sigma0', ctx.sigma0, ...
                                                  's0sq', ctx.s0sq, 'Cxx', ctx.Cxx), 'Verbose', 1
    figure(1); plot(res.sys(1).obs(1).t, res.sys(1).obs(1).w, '.'); drawnow;
    go = ctx.maxDev > 1e-3;                % or return nothing to continue
end
```

10 The engine, kernel and util tiers

What `fit` dispatches to and what it is built from. One line per public unit, from the `Contents.m` files (generated). `doc lsqvce.engine.<name>` opens the full page; the tier rule (HANDOVER, tiers.csv): the user tier calls engine, engine calls kernel and util, never upward.

10.1 lsqvce.engine — drivers, accumulator, loaders, sweeps, diagnostics

unit	what
`adjustNonlinear`	Gauss-Newton / Levenberg as a policy on the block path (R5 M2)
`batchSystem`	a batch source as one local-free streaming system (R5 M2)
`adjustRobust`	IRLS (Huber/IGG3/bisquare) as a policy on the block path (R5 M2)
`adjustBlock`	one GLS solve over the streaming block-arrow pipeline
`adjustVCEjoint`	Ellmer ten-step joint co-estimation with keyed arc factors
`applyRegularisation`	regularisation of the reduced system
`arcVarianceFactors`	per-arc variance factors
`BlockAccumulator`	block-arrow accumulation with per-system Schur pre-elimination
`checkPSD`	PSD sanity check
`collectResiduals`	residual collection per stream
`CommonPrior`	prior on the common block (regularisation as observations)
`contribution`	information contribution per group
`covComponent`	covariance component estimation
`covModel`	covariance spec -> whitening operators (per arc)
`datum`	datum defect detection and handling
`declaration`	validate a kind-A data struct and build its loaders (R5-D8)
`diagnose`	per-observation diagnostics (batch)
`diagnoseAccum`	diagnostics bridge from the accumulator
`ellipse`	confidence ellipse from a covariance

unit	what
`ftest`	F-test on variance factors
`gcvLambda`	GCV choice of the regularisation weight
`jointModelOpt`	joint-mode model -> loader UserOpt translation
`leverage`	leverage (hat-matrix diagonal), batch
`leverageBlocks`	leverage over blocks
`localDiagnostics`	diagnostics of the local blocks
`localColumns`	which (row, day, arc) owns each column of B (R5-D8)
`makeLoader`	day-builder -> Loader (built INSIDE fit from the data struct, R4-D4)
`mapreduce`	serial / parfor / spmd task runner with ordered reduce
`NormalEqSystem`	normal-equation backend
`outliers`	outlier listing from diagnostics
`psdVCE`	PSD amplitude + arc-factor VCE (Ellmer workhorse)
`QRSystem`	QR backend
`ReducedContribution`	reduced-normal contribution bookkeeping
`redundancy`	redundancy numbers (data pass)
`residuals`	second data pass: residuals and quadratic forms
`resolveNc`	common-parameter count derived from the systems
`satKeyOf`	satellite key (makeValidName) helper
`synthLoader`	synthetic block-arrow loader (engine-tier fixture; leaves in M4)
`sharedProbe`	shared Hutchinson probe generator (RNG contract)
`snoop`	block-native data snooping and downweighting
`stat`	statistics helper
`streamContribution`	contribution per stream
`systematic`	systematic-effect test
`traceEstimate`	stochastic trace estimate
`vce`	Helmert group variance components (exact
`runMeta`	immutable provenance record for Results.meta

10.2 lsqvce.kernel — the numerical kernels

unit	what
`ar2arma`	AR -> ARMA conversion
`armaACV`	ARMA autocovariance
`armaWhiten`	ARMA whitening filter (batched)
`crossCorr`	cross-correlation of residual streams
`dctI`	DCT-I transform
`decorrFactor`	decorrelation factor of a dense covariance
`levinsonDurbin`	Levinson-Durbin recursion
`psd2acv`	PSD -> autocovariance
`redundancyLags`	redundancy per lag without the dense W^-1
`separableWhiten`	separable (temporal x axis) whitening
`toeplitzCofactor`	Toeplitz cofactor construction
`varACV`	VAR autocovariance

unit	what
`varWhiten`	VAR whitening
`whitenessCorr`	whiteness (Ljung-Box) statistic
`whittle`	multichannel Levinson (Whittle) VAR fit (moved from engine, R5 M2)
`robustWeights`	IRLS weight factors from standardised residuals (R5 M2)

10.3 lsqvce.util — plumbing

unit	what
`callUser`	call a user function: outputs counted once, errors wrapped (R5 M1)
`getfielddef`	struct field with default
`toolboxVersion`	the version string from Contents.m (one source) (v5.2)
`onPass`	fire the per-pass inspection hook (Options.OnPass) (R5 M5)
`rowWeightKey`	the one key convention for per-arc robust weights (R5 M2)
`log`	toolbox-wide logging state
`mergeStruct`	struct merge (later wins)
`verbosity`	verbosity level resolution and formatting

11 Theory

`lsqvce_theory.pdf` (the R5 theory, 54 pages) is the authority for every formula the toolbox implements: the block-arrow adjustment and its two backends, the joint variance-component estimator (the per-lag redundancies, the PSD-domain law, the per-stream level and the arc factors), priors and Koch–Kusche, the diagnostics (Baarda, Pope, reliability, whiteness), IRLS and Gauss–Newton, and Appendix C, the option mapping as implemented. The literature: Ellmer (2018), Kvas (2019), Koch and Kusche (2002), Baarda (1968), Pope (1976).

12 Migration from 4.x to 5

Every 4.x declaration still fits; three things were added (priors, per-stream policy, `OnPass`), the option space was cut to 14 knobs + 5 groups, and every retired name has the successor below. Numbers may differ from 4.0 because the separate-mode estimator (an AR law fitted to a biased raw-residual ACF) is gone; the acceptance of its successor is the physical oracle, not equality to 4.0.

12.1 The migration table (from RELEASE_NOTES.md)

v4	v5
`Model="gls"`	`Estimate="none"` (default)
`Model="ellmer"`	`Estimate="noise"` (+ `Pool`, `ArcFactors=true` default)
`Model="vce"`	`Estimate="noise", Pool="stream", ArcFactors=false`

v4	v5
`Model="pooled"`	`Estimate="noise", Pool="all", ArcFactors=false`
`Robust="irls"` + `Influence`	`Robust="huber"|"igg3"|"bisquare"`
`Tuning`, `RobustMaxIter`, `RobustTol`	`RobustOpts.Tuning` / `.MaxIter` / `.Tol`
`MaxOuter`, `JointMaxIter`, `VceMaxIter`, `NonlinearMaxIter`	`MaxPasses`
`JointTol`, `OuterTol`, `VceTol`, `TolX`	`Tolerance`
`TolF`, `Lambda0`	`GaussNewton.TolF` / `.Lambda0`
`NumLags`	`Noise.Grid`
`PsdMode`	`Noise.Whitener` (`"toeplitz"` default, as before)
`MeanType`, `Smooth`, `MinRedundancy`, `SkipThreshold`	`Noise.MeanType` / `.Smooth` / `.FreezeRedundancy` / `.SkipThreshold`
`Parallel`, `DistributeDays`, `ParallelThreshold`	`Parallel.Mode` / `.DistributeDays` / `.Threshold`
`Alpha`, `Bonferroni`, `Beta`, `RMin`, `RWeak`, `Test`, `RobustCheck`, `Store`, `NWorst`	`Diagnostics.<same>`
`LogFid`, `LogFlag`, `ShowSeconds`, `ExportDir`, `plotPostSolve`, `plotPostModel`	`OnPass`; `R.plot(kind)` (`LogFile` was offered here and retired in R6 WP-101: it never opened a file)
`DatumB`	`data.priors = struct('L', B.', 'sigma', 1, 'name', 'datum')`
`Sigma0`, `Sparse`, `Verify`, `Progress`	retired
`GroupBy`, `ComponentsPerEpoch`, `MaxOrder`, `AcvTol`, `MaxLag`, `WhiteAlpha`	retired with the separate mode (groups are streams; the axis count comes from the builder; whiteness is fixed at K = 20, α = 0.05)
`Norm`, `C0`, `StepSchedule`, `AvgTail`	engine-tier only (`lsqvce.engine.adjustVCEjoint`)
`lsqvce.engine.adjust` / `robust` / `solveNonlinear` / `Accumulator` / `adjustVCE` / `Combine` / `BatchIndex` / `varComp` / `vceHutchinson`	gone; `fit` on the block path; `lsqvce.engine.adjustBlock` / `adjustRobust` / `adjustNonlinear` / `adjustVCEjoint` / `batchSystem` at engine tier
`R.source.verdict`, `history.nOuter`	`R.report().verdict`, `history.nIter`

Full ledger, decisions and round notes: `dev/HANDOVER.md` §12; theory: `lsqvce_theory.pdf` (R6 WP-101: it shipped under a second name in dev/ as well; that duplicate is gone – one document, one name).

13 Appendix A — error and warning identifiers

Every identifier the package raises, from `dev/docs_src/error_ids.csv` (the catalogue the audit keeps closed both ways: nothing raised that is not listed, nothing listed that is not raised).

identifier	where / what
`lsqvce:Options:badGrid`	Options — error — v4.0
`lsqvce:Options:badGroupField`	Options — error — v4.0
`lsqvce:Options:badOnPass`	Options — error — v4.0
`lsqvce:Options:badTarget`	Options — error — v4.0
`lsqvce:Options:badVerbose`	Options — error — v4.0
`lsqvce:Results:badPlotKind`	Results — error — v4.0
`lsqvce:Results:noAccumulator`	Results — error — v4.0

identifier	where / what
lsqvce:Results:noCovariance	Results — error — v4.0
lsqvce:Results:noHistory	Results — error — v4.0
lsqvce:Results:noVPv	Results — error — v4.0
lsqvce:Results:tableMismatch	Results — error — v4.0
lsqvce:accum:arcBadNc	engine.BlockAccumulator — error — v4.0
lsqvce:accum:arcBadNl	engine.BlockAccumulator — error — v4.0
lsqvce:accum:arcNotFinite	engine.BlockAccumulator — error — v4.0
lsqvce:accum:arcRowMismatch	engine.BlockAccumulator — error — v4.0
lsqvce:accum:badArc	engine.BlockAccumulator — error — v4.0
lsqvce:accum:badCxx	engine.BlockAccumulator — error — v4.0
lsqvce:accum:badLoader	engine.BlockAccumulator — error — v4.0
lsqvce:accum:badMeta	engine.BlockAccumulator — error — v4.0
lsqvce:accum:badPrior	engine.BlockAccumulator — error — v4.0
lsqvce:accum:badSpecs	engine.BlockAccumulator — error — v4.0
lsqvce:accum:badThresholds	engine.BlockAccumulator — error — v4.0
lsqvce:accum:badX	engine.BlockAccumulator — error — v4.0
lsqvce:accum:blockSchurFailed	engine.BlockAccumulator — warning — v4.0
lsqvce:accum:cancellation	engine.BlockAccumulator — warning — v4.0
lsqvce:accum:duplicateSystem	engine.BlockAccumulator — error — v4.0
lsqvce:accum:localNotRedundant	engine.BlockAccumulator — error — v4.0
lsqvce:accum:localSummarySkipped	engine.BlockAccumulator — warning — v4.0
lsqvce:accum:metaMismatch	engine.BlockAccumulator — error — v4.0
lsqvce:accum:noArcs	engine.BlockAccumulator — error — v4.0
lsqvce:accum:noDays	engine.BlockAccumulator — error — v4.0
lsqvce:accum:noStore	engine.BlockAccumulator — error — v4.0
lsqvce:accum:noSystems	engine.BlockAccumulator — error — v4.0
lsqvce:accum:notWholeEpochs	engine.private.arcBatchWhiten — error — v4.0
lsqvce:accum:pinvFallback	engine.BlockAccumulator — warning — v4.0
lsqvce:accum:priorNotPD	engine.BlockAccumulator — error — v4.0
lsqvce:accum:priorSize	engine.BlockAccumulator — error — v4.0
lsqvce:accum:unknownSystem	engine.BlockAccumulator — error — v4.0
lsqvce:adjustBlock:badCell	engine.private.normalizeSystems — error — v4.0
lsqvce:adjustBlock:badLoader	engine.private.normalizeSystems — error — v4.0
lsqvce:adjustBlock:badStruct	engine.private.normalizeSystems — error — v4.0
lsqvce:adjustBlock:badSystems	engine.private.normalizeSystems — error — v4.0
lsqvce:adjustBlock:noDays	engine.private.normalizeSystems — error — v4.0
lsqvce:adjustNonlinear:badX0	engine.adjustNonlinear — error — v4.0
lsqvce:adjustNonlinear:diverged	engine.adjustNonlinear — warning — v4.0
lsqvce:adjustNonlinear:maxIter	engine.adjustNonlinear — warning — v4.0
lsqvce:adjustNonlinear:nonFinite	engine.adjustNonlinear — error — v4.0
lsqvce:adjustRobust:collapsed	engine.adjustRobust — error — v4.0
lsqvce:adjustRobust:notConverged	engine.adjustRobust — warning — v4.0
lsqvce:adjustVCEjoint:arcMismatch	engine.adjustVCEjoint — error — v4.0
lsqvce:adjustVCEjoint:badAxisRows	engine.adjustVCEjoint — error — v4.0
lsqvce:adjustVCEjoint:badC0	engine.adjustVCEjoint — error — v4.0
lsqvce:adjustVCEjoint:badSigma2_0	engine.adjustVCEjoint — error — v4.0
lsqvce:adjustVCEjoint:localNotPD	engine.adjustVCEjoint — error — v4.0
lsqvce:adjustVCEjoint:needCfcn	engine.adjustVCEjoint — error — v4.0
lsqvce:adjustVCEjoint:noArcs	engine.adjustVCEjoint — error — v4.0

identifier	where / what
lsqvce:adjustVCEjoint:collapsed	engine.adjustVCEjoint — error — v6.1
lsqvce:adjustVCEjoint:notConverged	engine.adjustVCEjoint — warning — v4.0
lsqvce:adjustVCEjoint:arcFactorsDiverging	engine.adjustVCEjoint — warning — v5.3
lsqvce:adjustVCEjoint:notPD	engine.adjustVCEjoint — error — v4.0
lsqvce:adjustVCEjoint:skipThreshold	engine.adjustVCEjoint — error — v4.0
lsqvce:applyReg:badLambda	engine.applyRegularisation — error — v4.0
lsqvce:applyReg:badRegMatrix	engine.applyRegularisation — error — v4.0
lsqvce:applyReg:notPD	engine.applyRegularisation — error — v4.0
lsqvce:applyReg:sizeMismatch	engine.applyRegularisation — error — v4.0
lsqvce:applyReg:unknown	engine.applyRegularisation — error — v4.0
lsqvce:ar2arma:degenerate	kernel.ar2arma — error — v4.0
lsqvce:ar2arma:degenerateMA	kernel.ar2arma — warning — v4.0
lsqvce:ar2arma:tooShort	kernel.ar2arma — error — v4.0
lsqvce:ar2arma:unstable	kernel.ar2arma — warning — v4.0
lsqvce:arcVarianceFactors:arcMismatch	engine.arcVarianceFactors — error — v4.0
lsqvce:arcVarianceFactors:badMean	engine.arcVarianceFactors — warning — v4.0
lsqvce:arcVarianceFactors:clamped	engine.arcVarianceFactors — warning — v4.0
lsqvce:armaACV:nonStationary	kernel.armaACV — error — v4.0
lsqvce:armaACV:zeroLeadingAR	kernel.armaACV — error — v4.0
lsqvce:armaWhiten:acvTooShort	kernel.armaWhiten — error — v4.0
lsqvce:armaWhiten:badSigma	kernel.armaWhiten — error — v4.0
lsqvce:armaWhiten:sizeMismatch	kernel.armaWhiten — error — v4.0
lsqvce:armaWhiten:startupNotPD	kernel.armaWhiten — warning — v4.0
lsqvce:batchSystem:badMatrix	engine.batchSystem — error — v4.0
lsqvce:batchSystem:badSource	engine.batchSystem — error — v4.0
lsqvce:batchSystem:weightMatrix	engine.batchSystem — error — v4.0
lsqvce:callUser:badHandle	util.callUser — error — v4.0
lsqvce:callUser:threw	util.callUser — error — v4.0
lsqvce:callUser:tooFew	util.callUser — error — v4.0
lsqvce:checkPSD:badFit	engine.checkPSD — warning — v4.0
lsqvce:checkPSD:badModel	engine.checkPSD — error — v4.0
lsqvce:checkPSD:marginalFit	engine.checkPSD — warning — v4.0
lsqvce:checkPSD:unstable	engine.checkPSD — warning — v4.0
lsqvce:collectResiduals:badRes	engine.collectResiduals — error — v4.0
lsqvce:covComponent:badLag	engine.covComponent — error — v4.0
lsqvce:covComponent:lagTooLarge	engine.covComponent — error — v4.0
lsqvce:covModel:badCep	engine.covModel — error — v4.0
lsqvce:covModel:badPsdMode	engine.covModel — error — v4.0
lsqvce:covModel:badSigma	engine.covModel — error — v4.0
lsqvce:covModel:badWhitenMode	engine.covModel — error — v4.0
lsqvce:covModel:cepOnVar	engine.covModel — error — v5.3
lsqvce:covModel:degeneratePSD	engine.covModel — error — v4.0
lsqvce:covModel:negativePSD	engine.covModel — warning — v4.0
lsqvce:covModel:noACF	engine.covModel — error — v4.0
lsqvce:covModel:noAR	engine.covModel — error — v4.0
lsqvce:covModel:noCovForm	engine.covModel — error — v4.0
lsqvce:covModel:noFcn	engine.covModel — error — v4.0
lsqvce:covModel:noFilterForm	engine.covModel — error — v4.0
lsqvce:covModel:noPSD	engine.covModel — error — v4.0
lsqvce:covModel:noSigma	engine.covModel — error — v4.0
lsqvce:covModel:noType	engine.covModel — error — v4.0
lsqvce:covModel:noVAR	engine.covModel — error — v4.0
lsqvce:covModel:notWholeEpochs	engine.covModel — error — v4.0
lsqvce:covModel:psdSanitationEscape	engine.covModel — error — v4.0

identifier	where / what
`lsqvce:covModel:sigmaLength`	engine.covModel — error — v4.0
`lsqvce:covModel:unknownType`	engine.covModel — error — v4.0
`lsqvce:crossCorr:shortSeries`	kernel.crossCorr — warning — v4.0
`lsqvce:datum:BnotNull`	engine.datum — warning — v4.0
`lsqvce:datum:autoDetected`	engine.datum — warning — v4.0
`lsqvce:datum:badB`	engine.datum — error — v4.0
`lsqvce:datum:rankDeficientB`	engine.datum — error — v4.0
`lsqvce:datum:zeroN`	engine.datum — error — v4.0
`lsqvce:dctI:tooShort`	kernel.dctI — error — v4.0
`lsqvce:decorrFactor:notPD`	kernel.decorrFactor — mixed — v4.0
`lsqvce:diagnose:badPmatrix`	engine.diagnose — error — v4.0
`lsqvce:diagnose:badTest`	engine.diagnose — error — v4.0
`lsqvce:diagnose:batchMismatch`	engine.diagnose — error — v4.0
`lsqvce:diagnose:largeU`	engine.diagnose — warning — v4.0
`lsqvce:diagnose:nBatch`	engine.diagnose — error — v4.0
`lsqvce:diagnose:noRobustScale`	engine.diagnose — error — v4.0
`lsqvce:diagnoseAccum:needKeepLocal`	engine.diagnoseAccum — error — v4.0
`lsqvce:diagnoseAccum:noSystems`	engine.diagnoseAccum — error — v4.0
`lsqvce:ellipse:badPair`	engine.ellipse — error — v4.0
`lsqvce:ellipse:samePair`	engine.ellipse — error — v4.0
`lsqvce:fit:badArcs`	engine.declaration — error — v4.0
`lsqvce:fit:badEstimate`	fit — error — v4.0
`lsqvce:fit:badId`	engine.declaration — error — v4.0
`lsqvce:fit:badMatrix`	fit — error — v4.0
`lsqvce:fit:badParamColumns`	engine.declaration — error — v4.0
`lsqvce:fit:badPrior`	fit — error — v4.0
`lsqvce:fit:badScope`	engine.declaration — error — v4.0
`lsqvce:fit:builderKind`	engine.declaration — error — v4.0
`lsqvce:fit:duplicateSystem`	engine.declaration — error — v4.0
`lsqvce:fit:lawOnFixedStructure`	fit — error — v4.0
`lsqvce:fit:missingField`	engine.declaration;fit — error — v4.0
`lsqvce:fit:modelKind`	fit — error — v4.0
`lsqvce:fit:noCommon`	engine.declaration — error — v4.0
`lsqvce:fit:noDays`	engine.declaration — error — v4.0
`lsqvce:fit:noSystems`	engine.declaration — error — v4.0
`lsqvce:fit:notYet`	fit — error — v4.0
`lsqvce:fit:priorNotActive`	fit — warning — v4.0
`lsqvce:fit:uCount`	fit — error — v4.0
`lsqvce:fit:unknownField`	engine.declaration;fit — error — v4.0
`lsqvce:ftest:differentData`	engine.ftest — error — v4.0
`lsqvce:ftest:invalidPremise`	engine.ftest — warning — v4.0
`lsqvce:ftest:noRedundancy`	engine.ftest — error — v4.0
`lsqvce:ftest:notNested`	engine.ftest — error — v4.0
`lsqvce:ftest:zeroVariance`	engine.ftest — error — v4.0
`lsqvce:gcv:atLowerBound`	engine.gcvLambda — warning — v4.0
`lsqvce:gcv:atUpperBound`	engine.gcvLambda — warning — v4.0
`lsqvce:gcv:degenerate`	engine.gcvLambda — warning — v4.0
`lsqvce:gcvLambda:qzFallback`	engine.gcvLambda — warning — v4.0
`lsqvce:jointArcParts:sizeMismatch`	engine.private.jointArcParts — error — v4.0
`lsqvce:jointModelOpt:badC`	engine.jointModelOpt — error — v4.0
`lsqvce:jointModelOpt:badModel`	engine.jointModelOpt — error — v4.0
`lsqvce:jointModelOpt:badStream`	engine.jointModelOpt — error — v4.0
`lsqvce:leverage:outOfRange`	engine.leverage — warning — v4.0
`lsqvce:leverage:sizeMismatch`	engine.leverage — error — v4.0

identifier	where / what
`lsqvce:leverageBlocks:cNotPD`	engine.leverageBlocks — error — v4.0
`lsqvce:leverageBlocks:lNotPD`	engine.leverageBlocks — error — v4.0
`lsqvce:leverageBlocks:sizeMismatch`	engine.leverageBlocks — error — v4.0
`lsqvce:levinson:notPD`	kernel.levinsonDurbin — mixed — v4.0
`lsqvce:levinson:orderTooHigh`	kernel.levinsonDurbin — error — v4.0
`lsqvce:localDiagnostics:notFinite`	engine.localDiagnostics — error — v4.0
`lsqvce:log:noParallelQueue`	util.log — warning — v4.0
`lsqvce:makeKeyFcn:badGroupBy`	engine.private.makeKeyFcn — error — v4.0
`lsqvce:makeLoader:arcCount`	engine.makeLoader — error — v6.0
`lsqvce:makeLoader:badArc`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:badArcs`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:axisConflict`	engine.makeLoader — error — v5.3
`lsqvce:makeLoader:badDay`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:badDayCep`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:badPrior`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:badScope`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:badValidity`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:cepTwice`	engine.makeLoader — error — v5.3
`lsqvce:makeLoader:buildDay`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:dayShape`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:emptyDay`	engine.makeLoader — error — v6.0
`lsqvce:makeLoader:localPlan`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:ncConflict`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:nlConflict`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:noCep`	engine.makeLoader — warning — v5.3
`lsqvce:makeLoader:noArcs`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:noCov`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:noObsTypes`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:paramType`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:priorType`	engine.makeLoader — error — v4.0
`lsqvce:makeLoader:rowWeightSize`	engine.makeLoader — error — v4.0
`lsqvce:mapreduce:badTask`	engine.mapreduce — error — v4.0
`lsqvce:mapreduce:serialFallback`	engine.mapreduce — warning — v4.0
`lsqvce:normalEq:badMerge`	engine.NormalEqSystem — error — v4.0
`lsqvce:normalEq:badWidth`	engine.NormalEqSystem — error — v4.0
`lsqvce:normalEq:cancellation`	engine.NormalEqSystem — warning — v4.0
`lsqvce:normalEq:noData`	engine.NormalEqSystem — error — v4.0
`lsqvce:normalEq:noRedundancy`	engine.NormalEqSystem — error — v4.0
`lsqvce:normalEq:notSPD`	engine.NormalEqSystem — error — v4.0
`lsqvce:normalEq:sizeMismatch`	engine.NormalEqSystem — error — v4.0
`lsqvce:prior:badTarget`	engine.CommonPrior — error — v4.0
`lsqvce:probeMatrix:badDist`	engine.private.probeMatrix — error — v4.0
`lsqvce:psd2acv:aboveNyquist`	kernel.psd2acv — warning — v4.0
`lsqvce:psd2acv:nonPositiveVariance`	kernel.psd2acv — error — v4.0
`lsqvce:psd2acv:tooFewPoints`	kernel.psd2acv — error — v4.0
`lsqvce:psdVCE:arcMismatch`	engine.psdVCE — error — v4.0
`lsqvce:psdVCE:degenerateStart`	engine.psdVCE — warning — v4.0
`lsqvce:psdVCE:shapeMismatch`	engine.psdVCE — error — v4.0
`lsqvce:qr:badMerge`	engine.QRSystem — error — v4.0
`lsqvce:qr:badWidth`	engine.QRSystem — error — v4.0
`lsqvce:qr:cancellation`	engine.QRSystem — warning — v4.0
`lsqvce:qr:noData`	engine.QRSystem — error — v4.0
`lsqvce:qr:noRedundancy`	engine.QRSystem — error — v4.0
`lsqvce:qr:rankDeficient`	engine.QRSystem — error — v4.0

identifier	where / what
`lsqvce:qr:singularR`	engine.QRSystem — error — v4.0
`lsqvce:qr:sizeMismatch`	engine.QRSystem — error — v4.0
`lsqvce:reduced:notSquare`	engine.ReducedContribution — error — v4.0
`lsqvce:reduced:sizeMismatch`	engine.ReducedContribution — error — v4.0
`lsqvce:redundancy:hutchFailed`	engine.redundancy — warning — v4.0
`lsqvce:redundancy:hutchinsonExperimental`	engine.redundancy — warning — v4.0
`lsqvce:redundancy:identityViolated`	engine.redundancy — warning — v4.0
`lsqvce:redundancy:noStore`	engine.redundancy — error — v4.0
`lsqvce:redundancy:smallGroup`	engine.redundancy — warning — v4.0
`lsqvce:redundancy:spmdNoCollect`	engine.redundancy — warning — v4.0
`lsqvce:redundancyLags:badEbar`	kernel.redundancyLags — error — v4.0
`lsqvce:redundancyLags:badLocal`	kernel.redundancyLags — error — v4.0
`lsqvce:redundancyLags:badUiZ`	kernel.redundancyLags — error — v4.0
`lsqvce:redundancyLags:badW`	kernel.redundancyLags — error — v4.0
`lsqvce:residuals:badSol`	engine.residuals — error — v4.0
`lsqvce:residuals:noStore`	engine.residuals — error — v4.0
`lsqvce:residuals:sigma0Mismatch`	engine.residuals — warning — v4.0
`lsqvce:residuals:supersedesSol`	engine.residuals — warning — v4.0
`lsqvce:resolveNc:conflict`	engine.resolveNc — error — v4.0
`lsqvce:resolveNc:empty`	engine.resolveNc — error — v4.0
`lsqvce:resolveNc:missing`	engine.resolveNc — error — v4.0
`lsqvce:robustWeights:badInfluence`	kernel.robustWeights — error — v4.0
`lsqvce:robustWeights:badTuning`	kernel.robustWeights — error — v4.0
`lsqvce:runMeta:badArg`	engine.runMeta — error — v4.0
`lsqvce:separable:acvTooShort`	kernel.separableWhiten — error — v4.0
`lsqvce:separable:badSigma`	kernel.separableWhiten — error — v4.0
`lsqvce:separable:badSigmaShape`	kernel.separableWhiten — error — v4.0
`lsqvce:separable:epochNotPD`	kernel.separableWhiten — error — v4.0
`lsqvce:separable:notWholeEpochs`	kernel.separableWhiten — error — v4.0
`lsqvce:separable:sigmaLength`	kernel.separableWhiten — error — v4.0
`lsqvce:separable:startupNotPD`	kernel.separableWhiten — warning — v4.0
`lsqvce:sharedProbe:badProbes`	engine.sharedProbe — error — v4.0
`lsqvce:sharedProbe:notSquare`	engine.sharedProbe — error — v4.0
`lsqvce:snoop:badBatch`	engine.snoop — error — v4.0
`lsqvce:snoop:badIndex`	engine.snoop — error — v4.0
`lsqvce:snoop:breakdown`	engine.snoop — error — v4.0
`lsqvce:snoop:contaminated`	engine.snoop — warning — v4.0
`lsqvce:snoop:correlatedBatch`	engine.snoop — error — v4.0
`lsqvce:snoop:downdateFailed`	engine.snoop — warning — v4.0
`lsqvce:snoop:forcedBreakdown`	engine.snoop — warning — v4.0
`lsqvce:snoop:forcedContaminated`	engine.snoop — warning — v4.0
`lsqvce:snoop:forcedOverCap`	engine.snoop — warning — v4.0
`lsqvce:snoop:lowRedundancy`	engine.snoop — warning — v4.0
`lsqvce:snoop:maxReject`	engine.snoop — warning — v4.0
`lsqvce:snoop:rejectFraction`	engine.snoop — warning — v4.0
`lsqvce:snoop:tooContaminated`	engine.snoop — error — v4.0
`lsqvce:stat:badDof`	engine.stat — error — v4.0
`lsqvce:stat:badProb`	engine.stat — error — v4.0
`lsqvce:stat:sizeMismatch`	engine.stat — error — v4.0
`lsqvce:streamContribution:badSol`	engine.streamContribution — error — v4.0
`lsqvce:streamContribution:cNotPD`	engine.streamContribution — error — v4.0
`lsqvce:streamContribution:noArcs`	engine.streamContribution — error — v4.0
`lsqvce:streamContribution:noStore`	engine.streamContribution — error — v4.0
`lsqvce:synthLoader:badArc`	engine.synthLoader — error — v4.0

identifier	where / what
`lsqvce:synthLoader:noNoiseModel`	engine.synthLoader — error — v4.0
`lsqvce:systematic:badCovariate`	engine.systematic — error — v4.0
`lsqvce:systematic:lbUnreliable`	engine.systematic — warning — v4.0
`lsqvce:systematic:noResiduals`	engine.systematic — error — v4.0
`lsqvce:systematic:tooFew`	engine.systematic — warning — v4.0
`lsqvce:toeplitzCofactor:badVariance`	kernel.toeplitzCofactor — warning — v4.0
`lsqvce:toeplitzCofactor:empty`	kernel.toeplitzCofactor — error — v4.0
`lsqvce:toeplitzCofactor:notACV`	kernel.toeplitzCofactor — warning — v4.0
`lsqvce:traceEstimate:badOperator`	engine.traceEstimate — error — v4.0
`lsqvce:traceEstimate:needSize`	engine.traceEstimate — error — v4.0
`lsqvce:traceEstimate:notSquare`	engine.traceEstimate — error — v4.0
`lsqvce:traceEstimate:sizeMismatch`	engine.traceEstimate — error — v4.0
`lsqvce:util:log:badKey`	util.log — error — v4.0
`lsqvce:varACV:badA`	kernel.varACV — error — v4.0
`lsqvce:varACV:notStationary`	kernel.varACV — error — v4.0
`lsqvce:varWhiten:badA`	kernel.varWhiten — error — v4.0
`lsqvce:varWhiten:badSigma`	kernel.varWhiten — error — v4.0
`lsqvce:varWhiten:badStartup`	kernel.varWhiten — error — v4.0
`lsqvce:varWhiten:notWholeEpochs`	kernel.varWhiten — error — v4.0
`lsqvce:vce:badAccFcn`	engine.vce — error — v4.0
`lsqvce:vce:badSigma0`	engine.vce — error — v4.0
`lsqvce:vce:groupMismatch`	engine.vce — error — v4.0
`lsqvce:vce:hutchGroupMismatch`	engine.vce — error — v4.0
`lsqvce:vce:lowRedundancy`	engine.vce — warning — v4.0
`lsqvce:vce:notConverged`	engine.vce — warning — v4.0
`lsqvce:verbosity:badLevel`	util.verbosity — error — v4.0
`lsqvce:verbosity:badTag`	util.verbosity — error — v4.0
`lsqvce:whittle:badG`	kernel.whittle — error — v4.0
`lsqvce:whittle:notPD`	kernel.whittle — error — v4.0